



MANUAL TÉCNICO PIG LATIN



ANTLR





Tecnologías utilizadas

El proyecto PigLatin está desarrollado en Java 21, utilizando Maven como herramienta de gestión de dependencias y construcción. A continuación se detallan las principales tecnologías y librerías empleadas:

ANTLR 4

Se utiliza para la definición y generación del parser y lexer del lenguaje. Permite construir gramáticas y generar automáticamente código Java para el análisis sintáctico.

Versión: 4.13.1

Plugins: antlr4-maven-plugin con soporte para **visitor** y **listener**.

JavaFX

Framework gráfico para construir la interfaz de usuario del proyecto.

javafx-controls (componentes gráficos como botones, tablas, etc.)

Versión: 21.0.7

Graphviz-Java

Librería para generar y manipular grafos, utilizada en la representación visual de árboles de derivación y estructuras sintácticas.

Versión: 0.18.1

SLF4J (Simple Logging Facade for Java)

API de logging que provee una capa de abstracción para el manejo de registros. Se incluye junto con slf4j-simple para evitar advertencias de configuración en Graphviz.

Versión: 1.7.36

Herramientas de construcción

Maven

Sistema de gestión de proyectos y dependencias. Se utiliza para compilar, empaquetar y ejecutar el proyecto.

Configuración de compilador: maven.compiler.source y maven.compiler.target en versión 21.
Codificación: UTF-8.

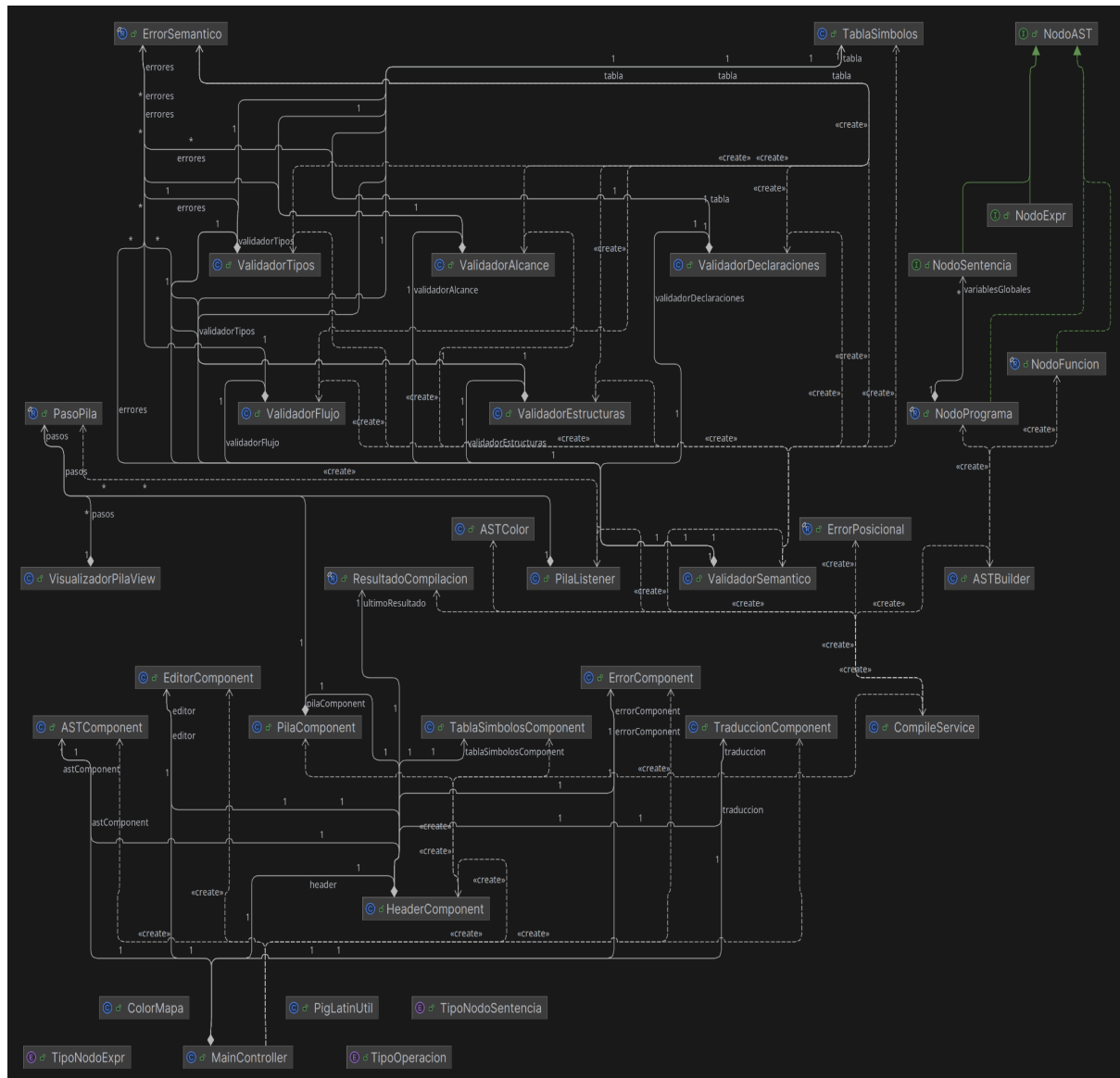
JavaFX Maven Plugin

Plugin que facilita la ejecución de aplicaciones JavaFX directamente desde Maven.

Versión: 0.0.8

Clase principal: com.example.piglatin.Main

diagrama general de clases





gramatica

Programa -> SeccionVariables? SeccionFunciones? SeccionMain FIN_PROGRAMA PUNTOCOMA EOF

SeccionVariables -> VARIABLES MAYOR DeclaracionVar*

SeccionFunciones -> MUNERA MAYOR Funcion*

SeccionMain -> MAIOR MAYOR Sentencia*

DeclaracionVar -> Variable

| Arreglo

| StructDef

| StructInstancia

Variable -> ESTO ID DOSPUNTOS TipoPrimitivo Expr? PUNTOCOMA
| ESTO ID DOSPUNTOS (VERUM | FALSUS) PUNTOCOMA

Tipo -> TipoPrimitivo | ID

TipoPrimitivo -> NUMERUS | TEXTUM | DECIMALIS | LITTERA

Arreglo -> SERIES ID CORCH_A INT CORCH_C DOSPUNTOS Tipo (LLAVE_A ListaExpr LLAVE_C)? PUNTOCOMA
| SERIES ID CORCH_A INT CORCH_C DOSPUNTOS LLAVE_A ListaExpr LLAVE_C PUNTOCOMA

ListaExpr -> Expr (COMA Expr)*

StructDef -> STRUCTURA ID LLAVE_A CampoStruct* LLAVE_C FINIS PUNTOCOMA

CampoStruct -> ESTO ID DOSPUNTOS Tipo PUNTOCOMA

StructInstancia -> ESTO ID DOSPUNTOS ID LiteralStruct

LiteralStruct -> LLAVE_A AsignacionCampo (COMA AsignacionCampo)* LLAVE_C

AsignacionCampo -> ID DOSPUNTOS Expr



Funcion -> ACTIO ID PAR_A ListaParametros? PAR_C LLAVE_A BloqueVariables? Sentencia* LLAVE_C FINIS PUNTOCOMA
| RATIO Tipo ID PAR_A ListaParametros? PAR_C LLAVE_A BloqueVariables? Sentencia* LLAVE_C FINIS PUNTOCOMA

ListaParametros -> Parametro (COMA Parametro)*

Parametro -> ESTO ID DOSPUNTOS Tipo

BloqueVariables -> VARIABLES CORCH_A DeclaracionVar* CORCH_C

Sentencia -> DeclaracionVar

- | Asignacion
- | AsignacionStructLiteral
- | Condicional
- | CicloDum
- | CicloFacere
- | CicloPer
- | Retorno
- | Lectura
- | Escritura
- | InterrupcionCiclo
- | LlamadaFuncion PUNTOCOMA
- | IncrementoDecremento

IncrementoDecremento -> (Referencia (INC | DEC) | (INC | DEC) Referencia) PUNTOCOMA

Asignacion -> Referencia ASIGNAR Expr PUNTOCOMA

AsignacionStructLiteral -> Referencia ASIGNAR LiteralStruct

Referencia -> ID

- | Referencia PUNTO ID
- | Referencia CORCH_A Expr CORCH_C



Condicional -> SI PAR_A Expr PAR_C BloqueSentencias RamaAliter* RamaElse? FINIS PUNTOCOMA

BloqueSentencias -> LLAVE_A Sentencia* LLAVE_C

RamaAliter -> ALITER PAR_A Expr PAR_C BloqueSentencias

RamaElse -> ALITER BloqueSentencias

CicloDum -> DUM PAR_A Expr PAR_C LLAVE_A Sentencia* LLAVE_C FINIS PUNTOCOMA

CicloFacere -> FACERE LLAVE_A Sentencia* LLAVE_C DUM PAR_A Expr PAR_C PUNTOCOMA

CicloPer -> PER PAR_A Variable Expr PUNTOCOMA Incremento PAR_C LLAVE_A Sentencia* LLAVE_C

Incremento -> Referencia (INC | DEC)
| Referencia ASIGNAR Expr

InterrupcionCiclo -> (PERGE | INTERRUMPE) PUNTOCOMA

Retorno -> REDDERE Expr PUNTOCOMA

Lectura -> ID? LEER

Escritura -> ESCRIBIR Expr (ESCRIBIR Expr)* PUNTOCOMA

LlamadaFuncion -> ID PAR_A ListaArgumentos? PAR_C

ListaArgumentos -> Expr (COMA Expr)*

Expr -> PAR_A Expr PAR_C

| (INC | DEC) ID

| ID (INC | DEC)

| NON Expr

| Expr (MULT | DIV) Expr

| Expr (MAS | MENOS) Expr

| Expr (MENOR | MAYOR | MENIG | MAYIG) Expr

| Expr (IGUAL | DISTINTO) Expr

| Expr AND Expr

| Expr OR Expr



| LlamadaFuncion

| Referencia

| INT

| FLOAT

| STRING

| CHAR

| VERUM

| FALSUS



ASTBluider (clase principal)

Extiende **LatinusParserBaseVisitor**, lo que significa que implementa un visitor para recorrer el árbol sintáctico generado por ANTLR.

Su objetivo es construir un Árbol de Sintaxis Abstracta (AST) a partir del árbol de análisis concreto.

Cada método visitXxx corresponde a una regla de la gramática y devuelve un objeto **NodoAST** especializado.

Sección Programa

visitPrograma:

Recorre las secciones de variables y funciones, construyendo listas de nodos (**NodoSentencia** y **NodoFuncion**).

Llama a **construirCuerpo** para obtener las sentencias del bloque principal.

Devuelve un **NodoPrograma** que encapsula todo el programa.

Declaraciones

visitDeclaracionVar: visita una declaración de variable, arreglo o struct.

visitVariablePrimitiva: crea un nodo de declaración de variable con tipo primitivo y valor opcional.

visitVariableBooleana: construye una variable booleana (VERUM o FALSUS).

visitArregloTipado y visitArregloBooleano: generan nodos de arreglos, con valores iniciales si existen.

visitStructDef: define una estructura con sus campos.

visitStructInstancia: instancia una estructura con valores iniciales.

construirMapaCampos: método auxiliar que convierte los campos de un literal de struct en un **Map<String, NodoExpr>**.

Funciones

visitFuncionSinRetorno y visitFuncionConRetorno: crean nodos de función con o sin tipo de retorno.

construirFuncion:

Procesa parámetros, variables locales y sentencias.

Devuelve un **NodoFuncion** con toda la información.

Sentencias

visitSentencia: decide qué tipo de sentencia es (llamada, asignación, condicional, etc.).

construirCuerpo: recorre una lista de sentencias y las convierte en nodos AST.

visitAsignacion: construye una asignación simple.

visitAsignacionStructLiteral: asigna un literal de struct a una referencia.

visitReferenciaBase, visitAccesoAtributo, visitAccesoArray: crean nodos de referencia a variables, atributos o elementos de arreglo.

visitCondicional: arma un nodo condicional con ramas if, aliter y else.

visitCicloDum, visitCicloFacere, visitCicloPer: construyen nodos de ciclos (while, do-while, for).

construirIncremento: transforma incrementos/decrementos en asignaciones equivalentes.

visitRetorno: crea un nodo de retorno.

visitLectura: nodo para leer entrada.

visitEscritura: nodo para imprimir valores.



visitInterrupcionCiclo: nodo para break o continue.

Expresiones

visitExprParentesis: devuelve la expresión interna.

visitExprIncDecPrefijo / visitExprIncDecPostfijo: crean expresiones unarias de incremento/decremento.

visitExprNegacion: expresión unaria de negación lógica.

visitExprMulDiv, visitExprSumaResta, visitExprRelacional, visitExprIgualdad: construyen expresiones binarias con operadores aritméticos o relacionales.

visitExprAnd, visitExprOr: expresiones lógicas binarias.

construirBinaria: método auxiliar para crear nodos binarios.

visitExprLlamada / visitLlamadaFuncion: crean nodos de llamada a función con argumentos.

visitExprReferencia: devuelve una referencia.

visitExprEntero, visitExprDecimal, visitExprTexto, visitExprCaracter: literales básicos.

visitExprVerum, visitExprFalsus: literales booleanos.

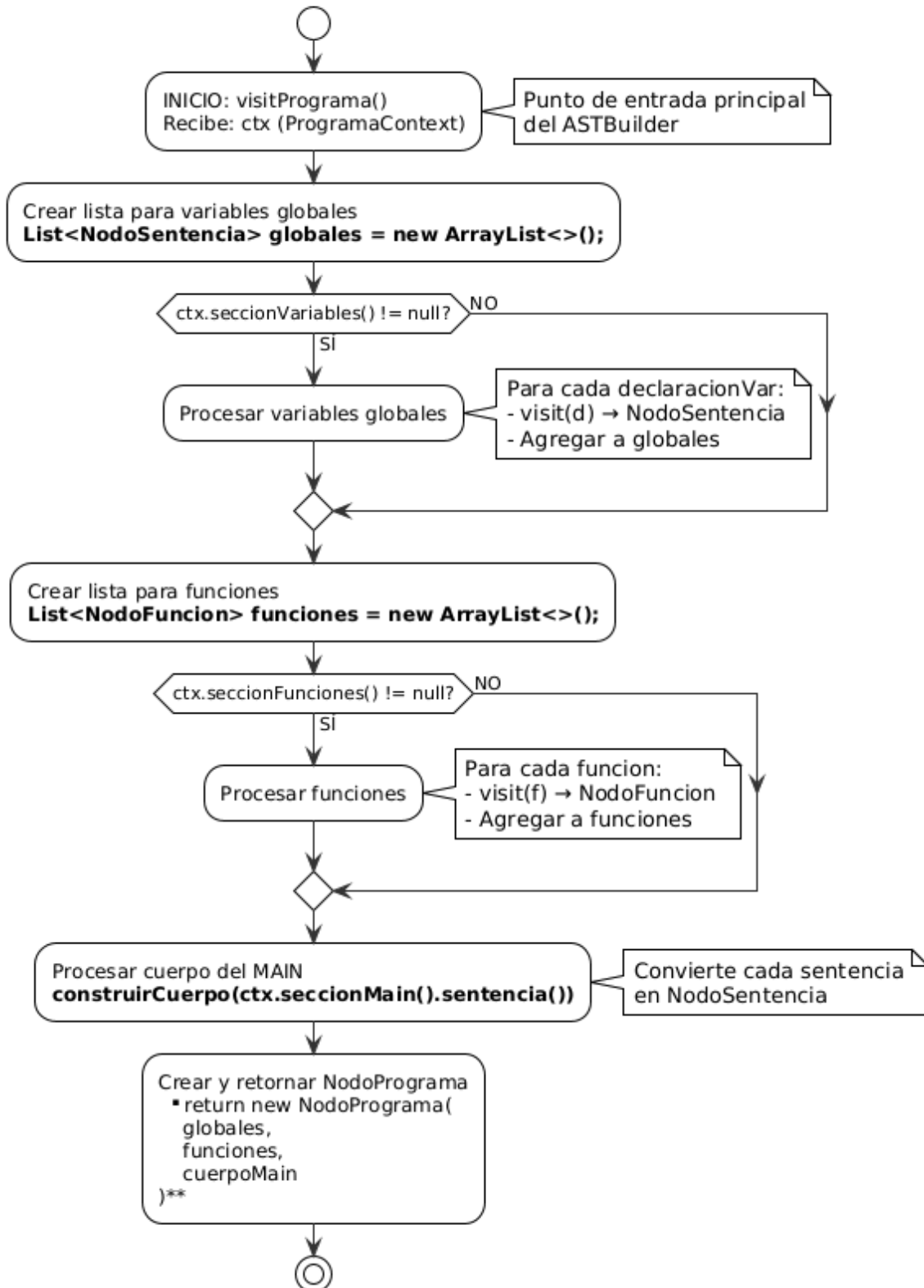
Utilidad

linea: obtiene el número de línea del contexto ANTLR para asociar cada nodo AST con su posición en el código fuente.



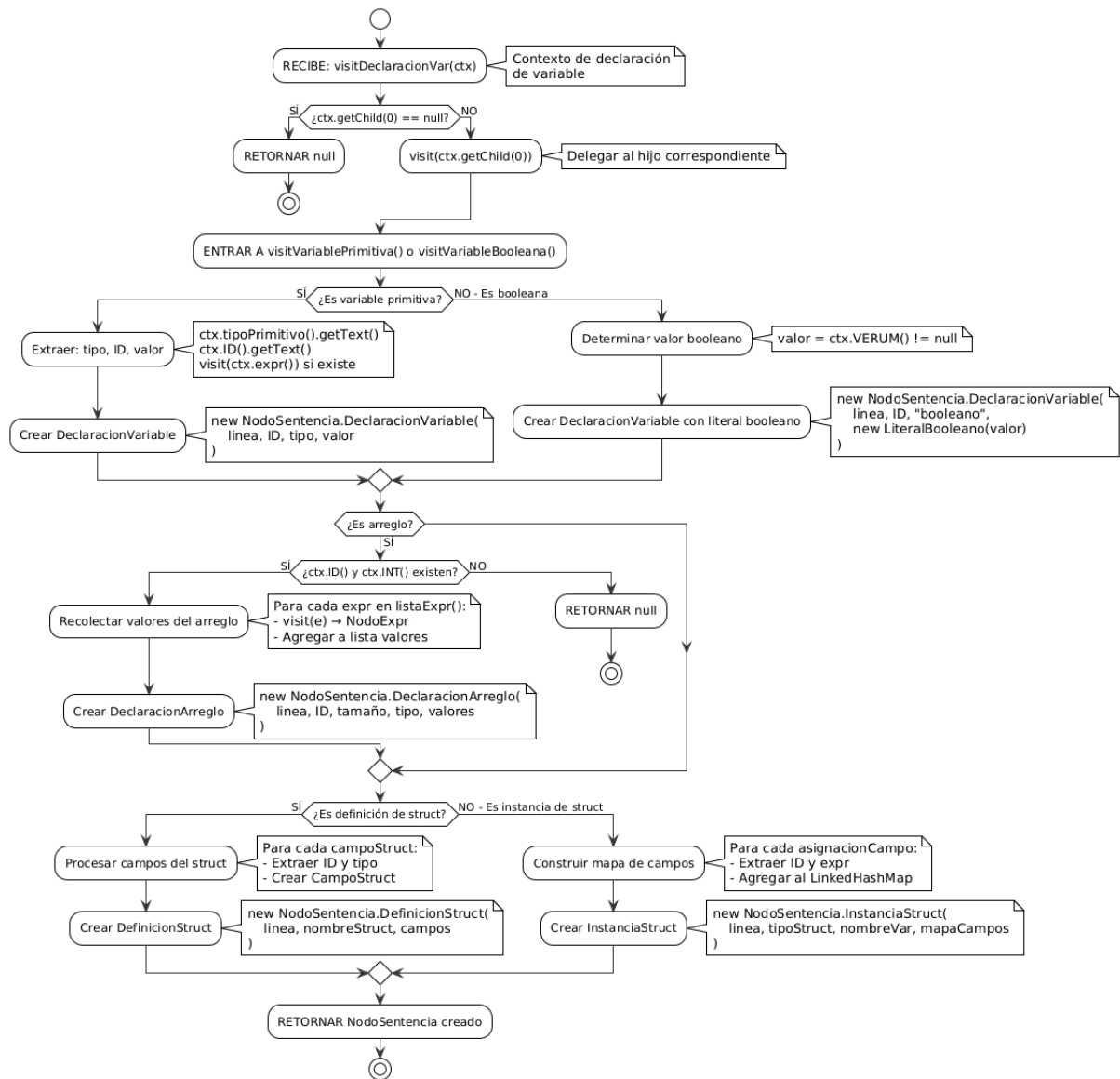
FLUJO DEL ASTBuild

Flujo Fase 1: Estructura Principal - ASTBuilder



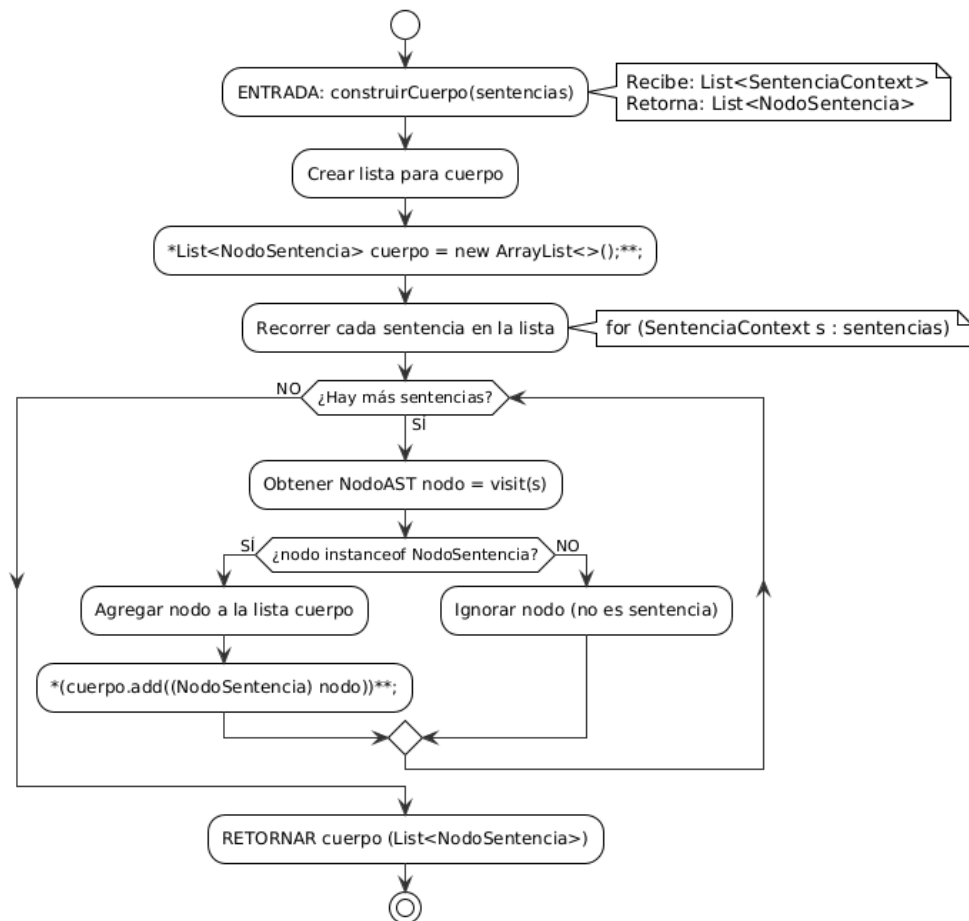


Flujo Fase 2: Procesamiento de Declaraciones - ASTBuilder





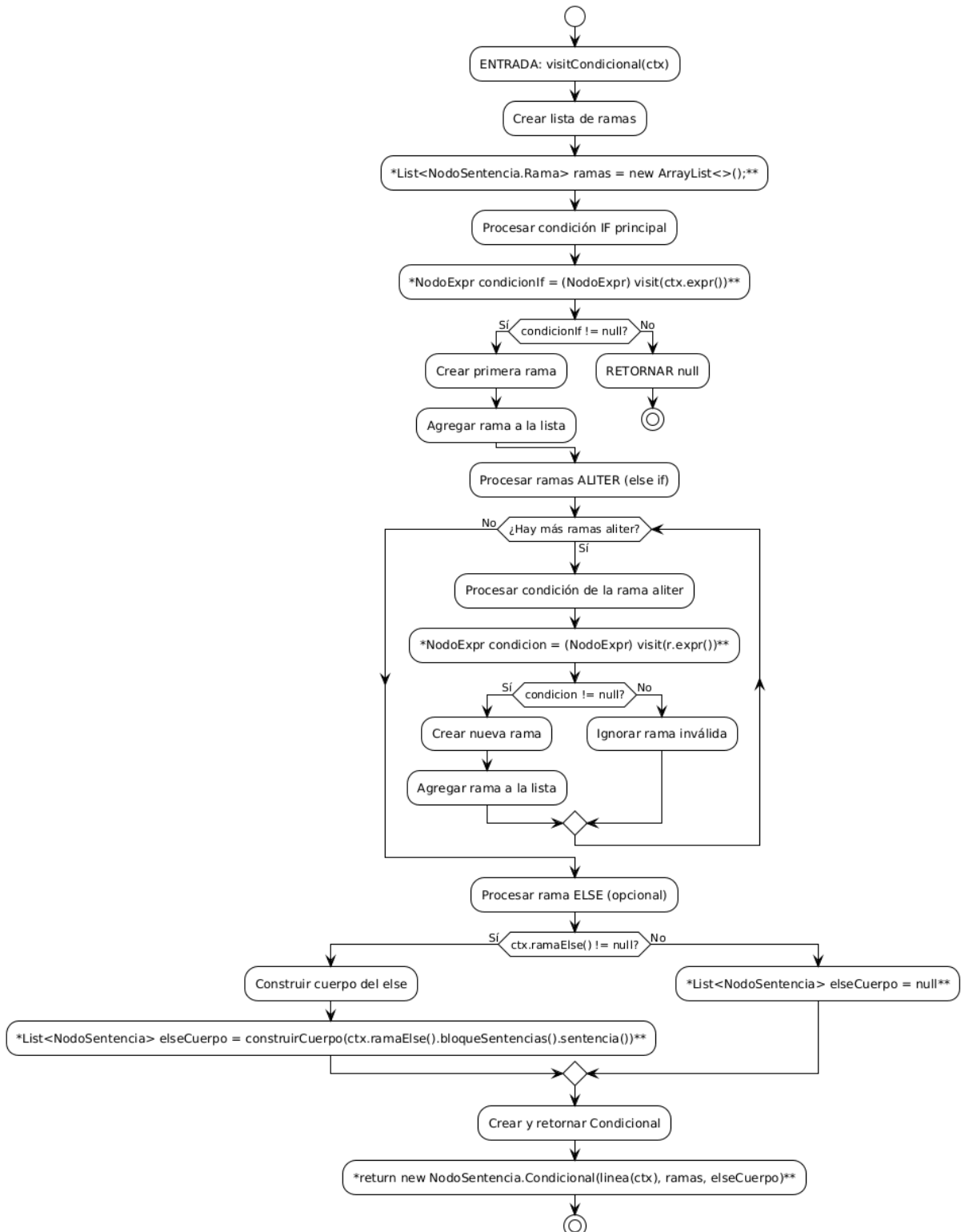
Flujo Fase 3: Construcción del Cuerpo de Sentencias - ASTBuilder





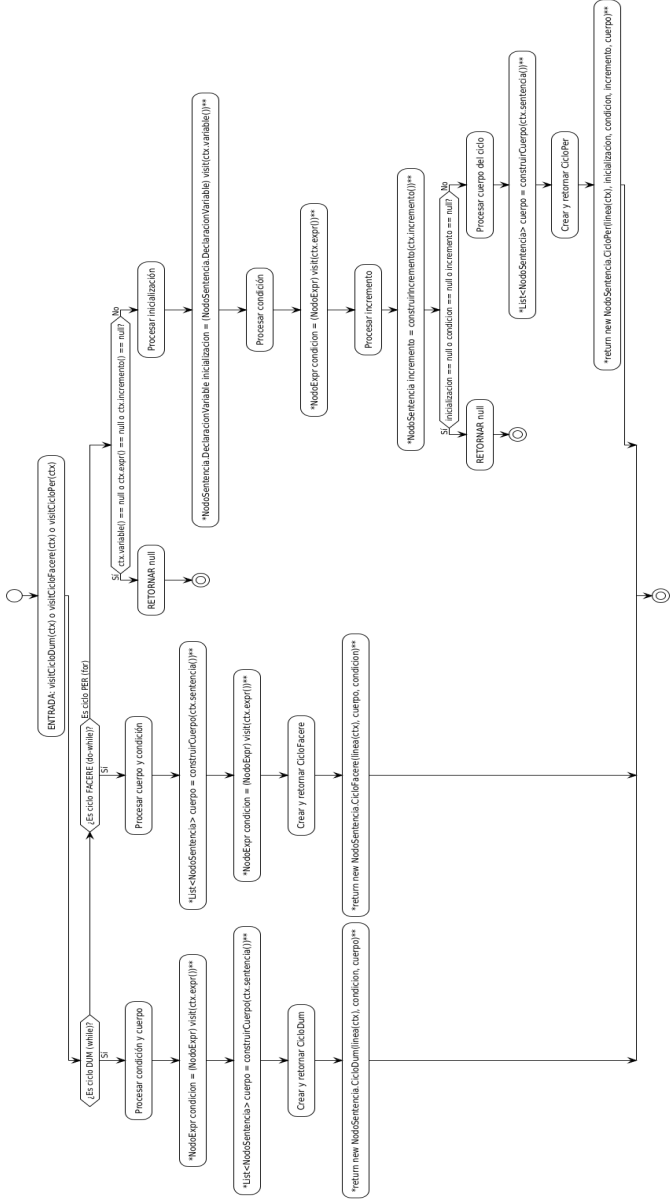
divisiones de la fase 3

Flujo: Sentencia Condicional - ASTBuilder



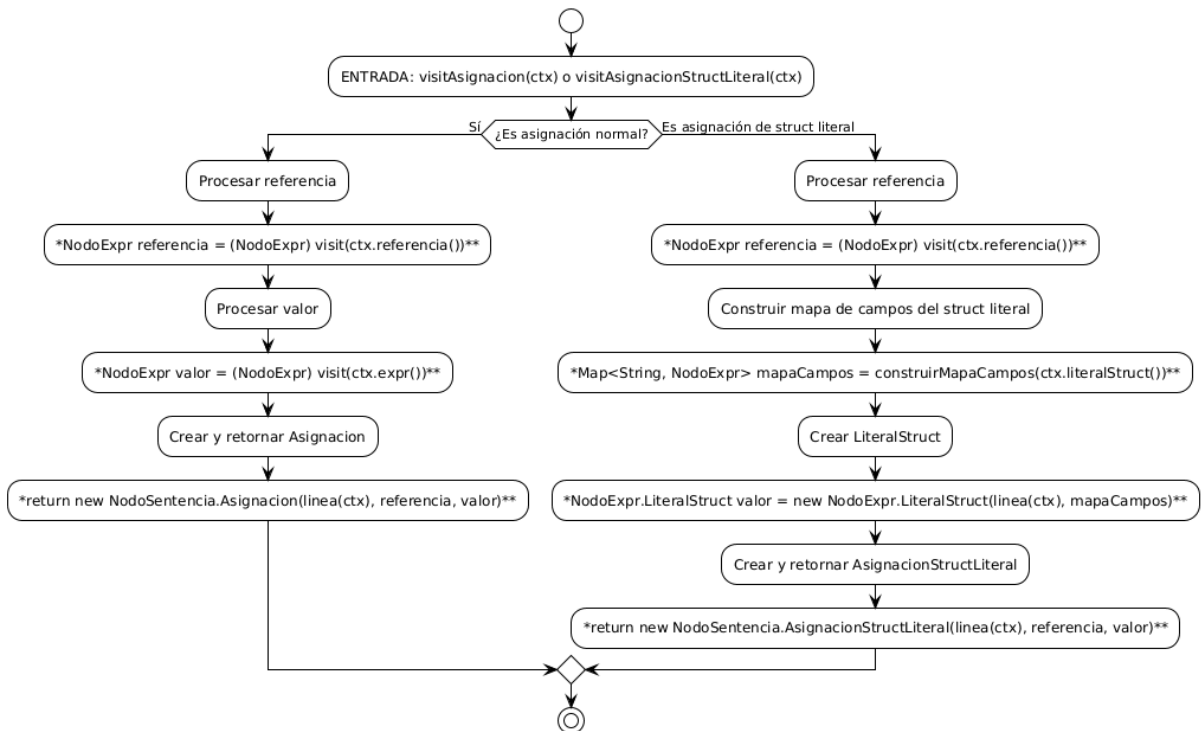


Flujo: Sentencias de Ciclos - ASTBuilder

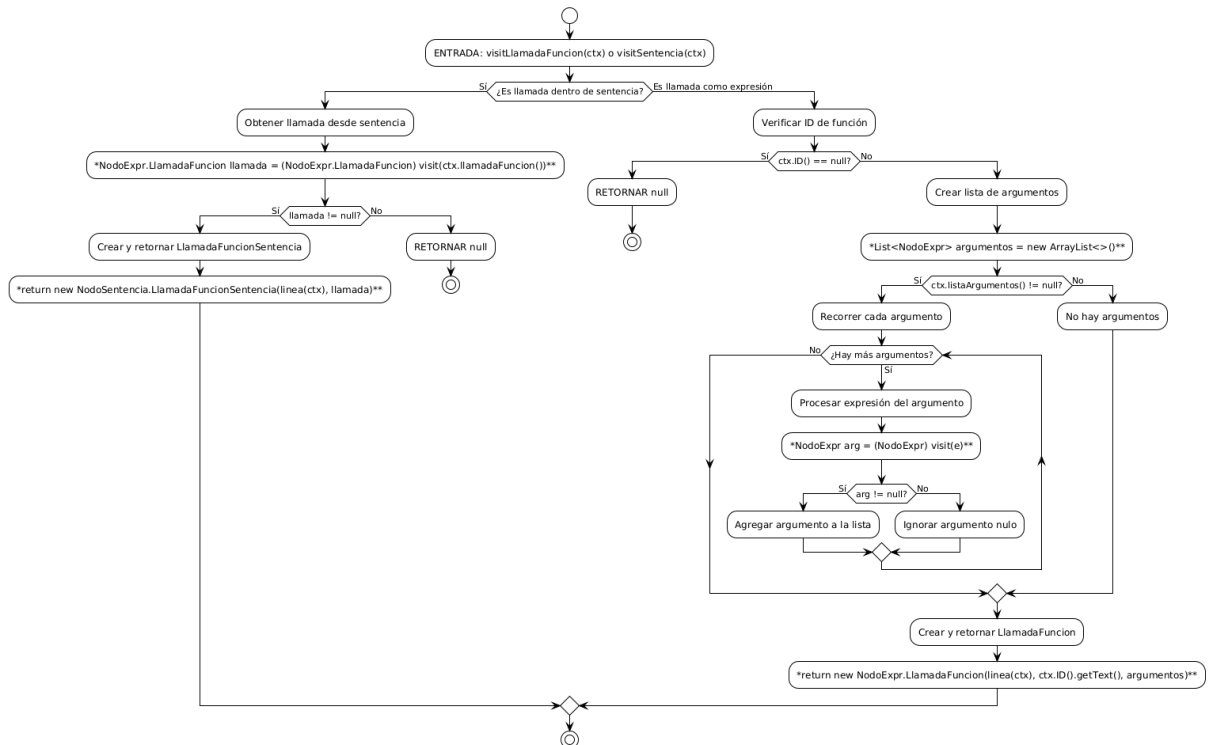




Flujo: Sentencias de Asignación - ASTBuilder

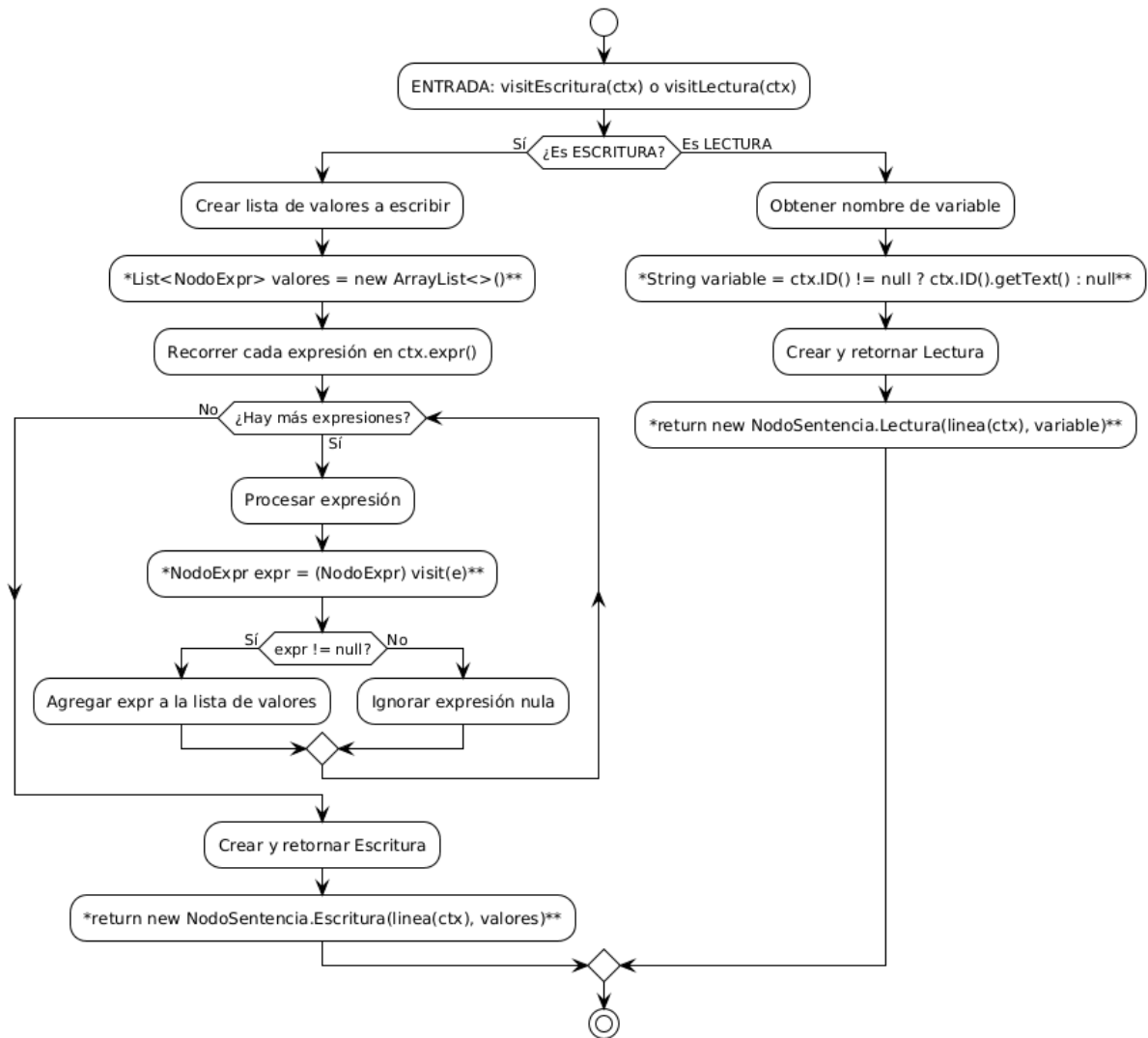


Flujo: Llamadas a Función - ASTBuilder



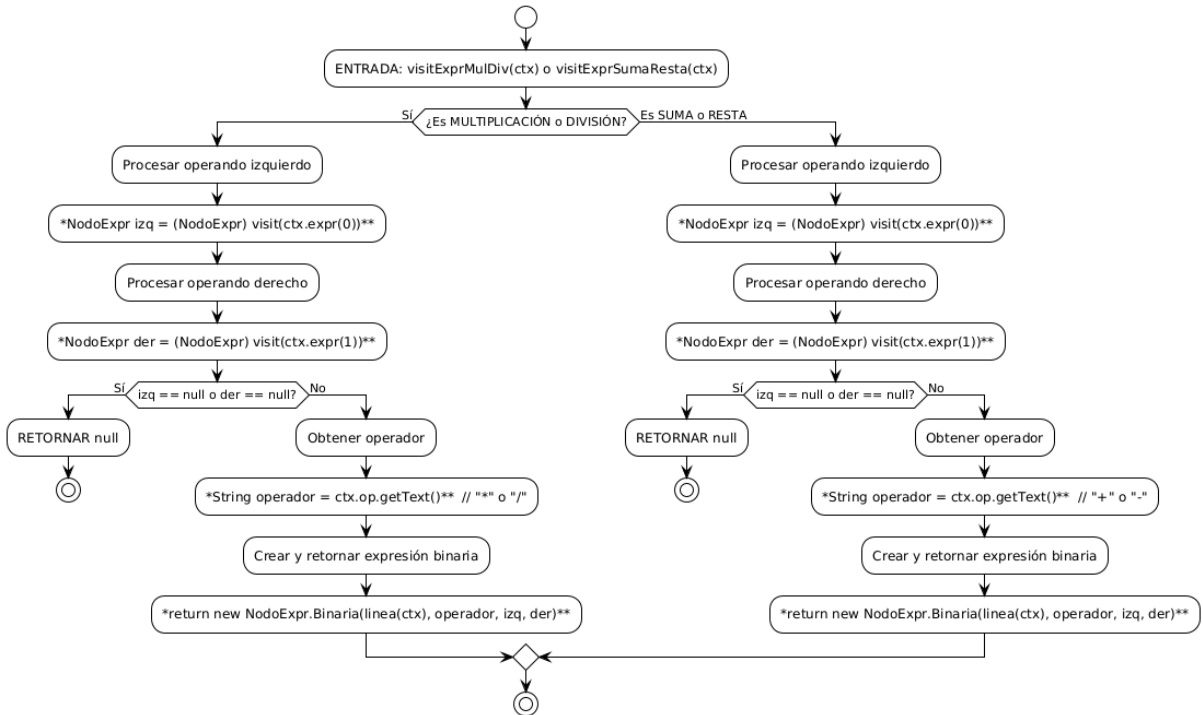


Flujo: Escritura y Lectura - ASTBuilder

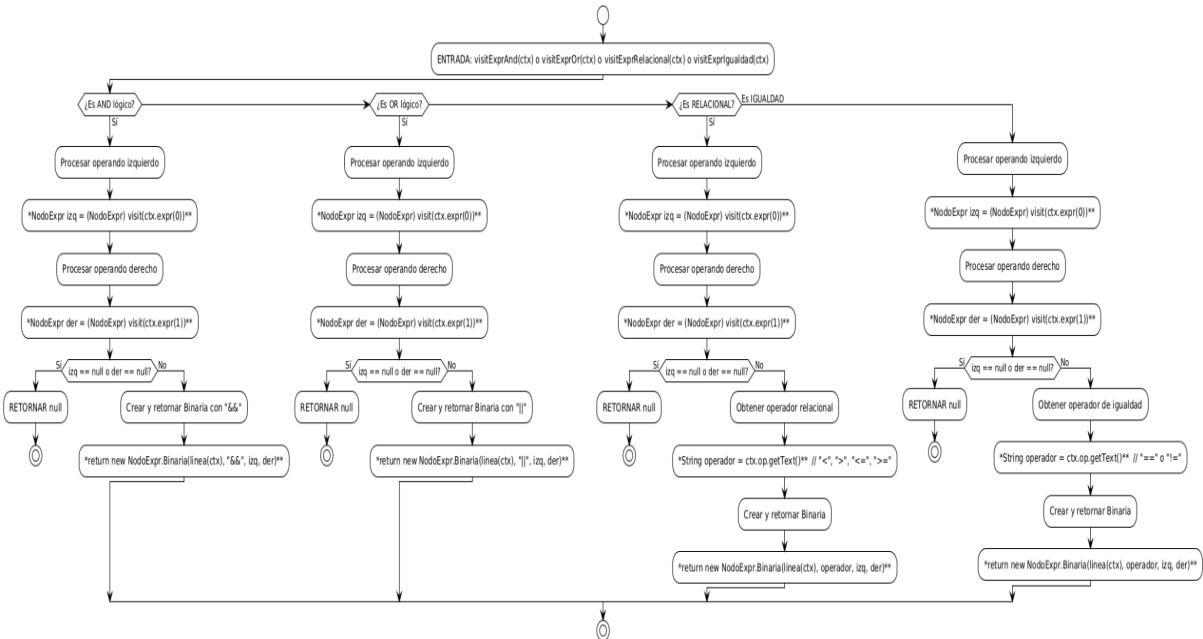




Flujo: Expresiones Aritméticas - ASTBuilder



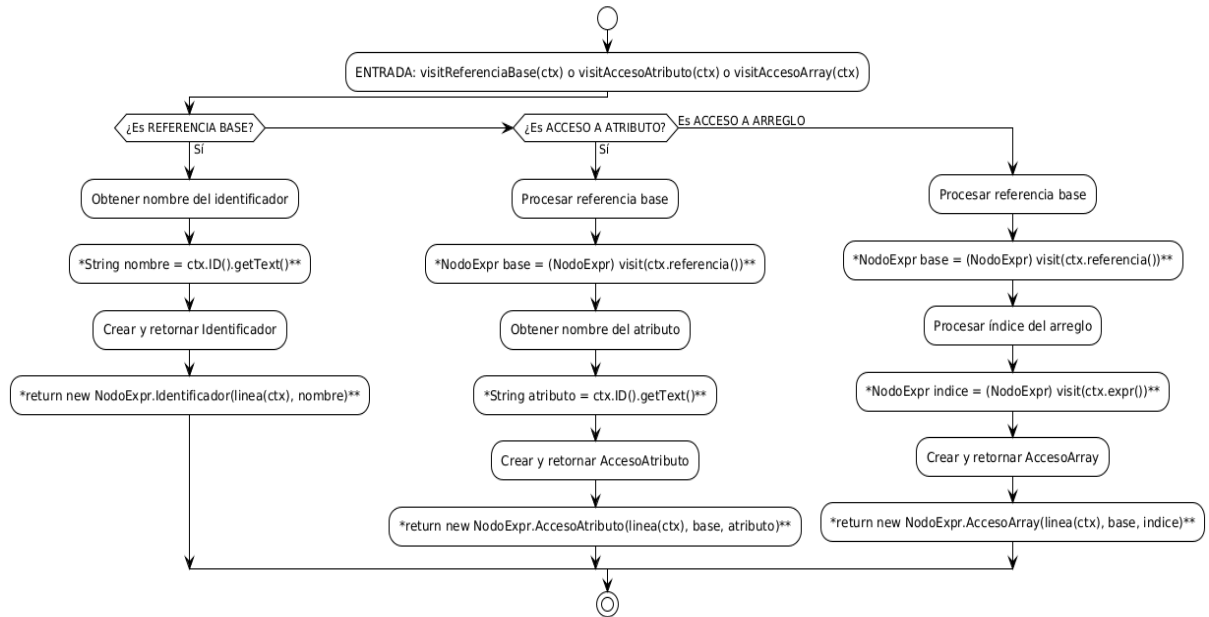
Flujo: Expresiones Lógicas y Relacionales - ASTBuilder





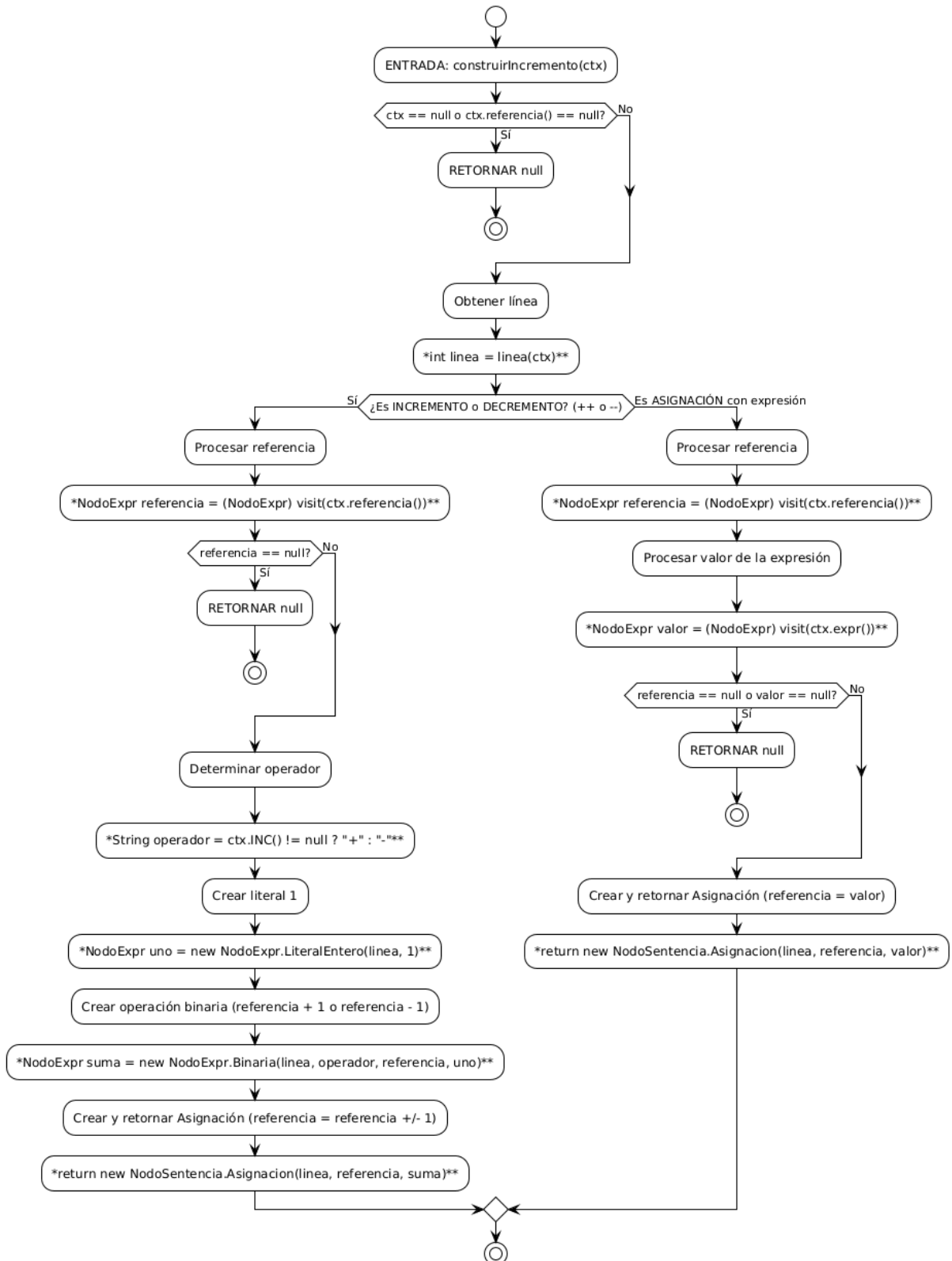
complejos

Flujo: Manejo de Referencias (Identificadores, Atributos y Arreglos)



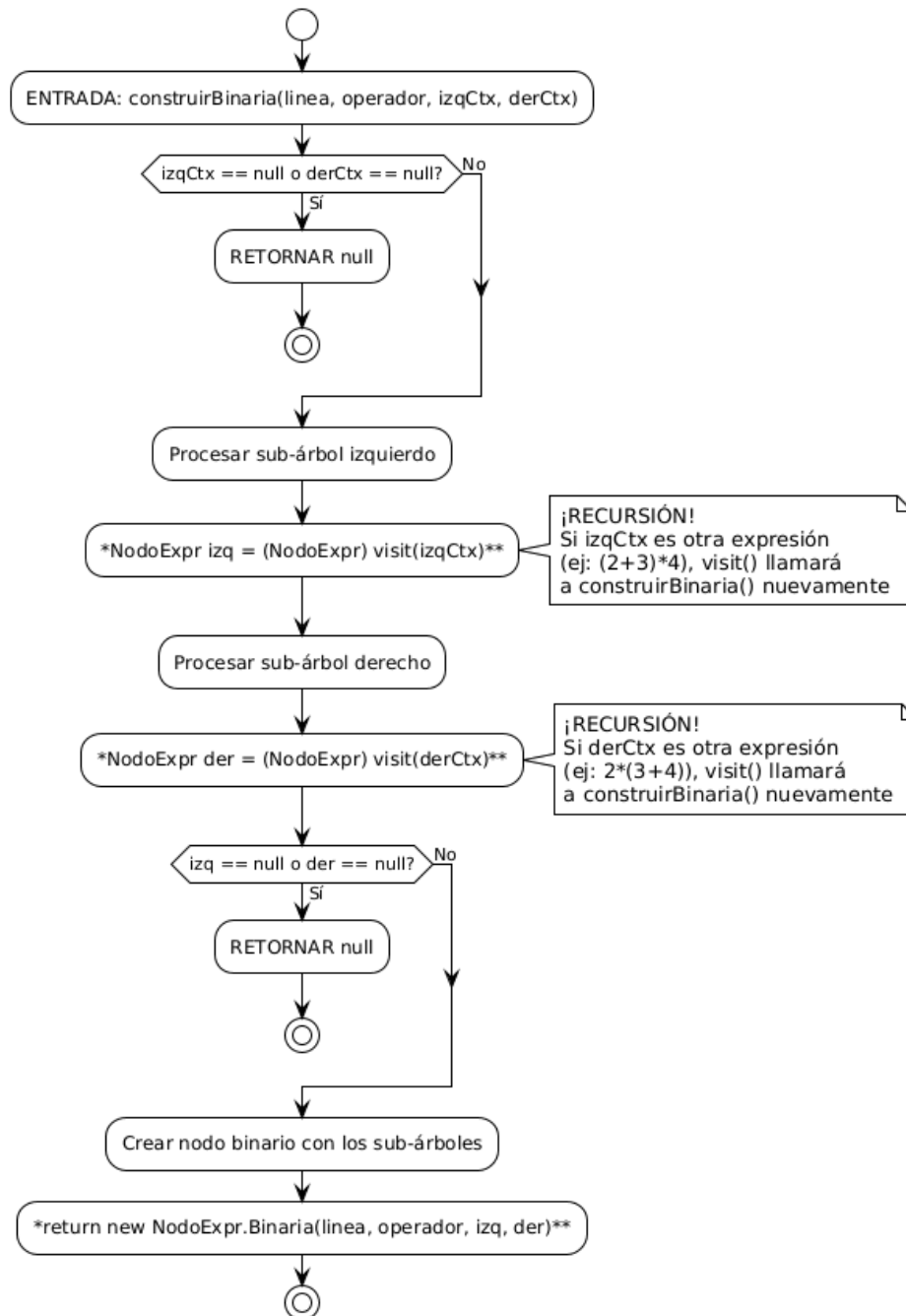


Flujo: Construcción de Incremento en Ciclo PER



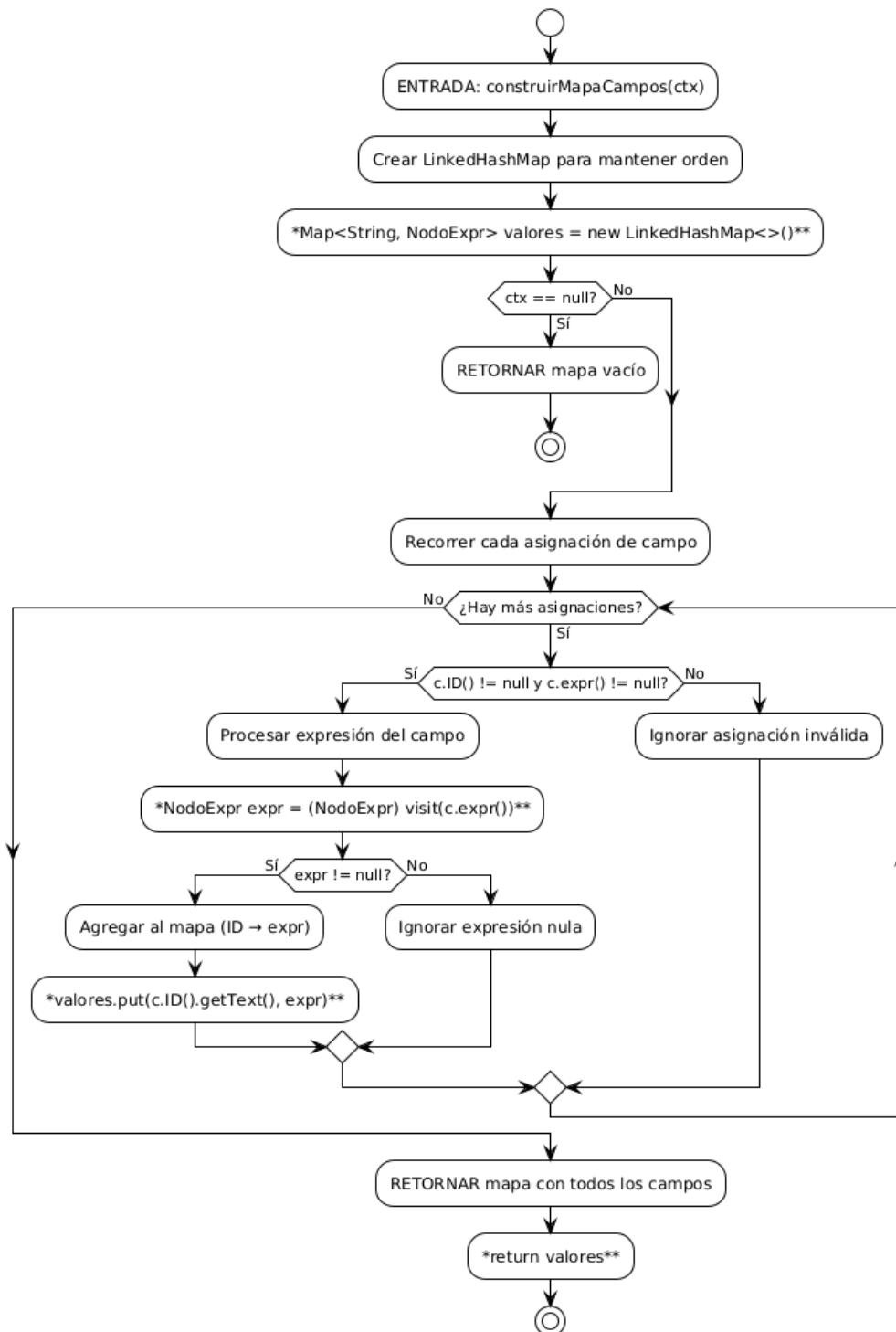


Flujo: construirBinaria() - Manejo de Expresiones Anidadas



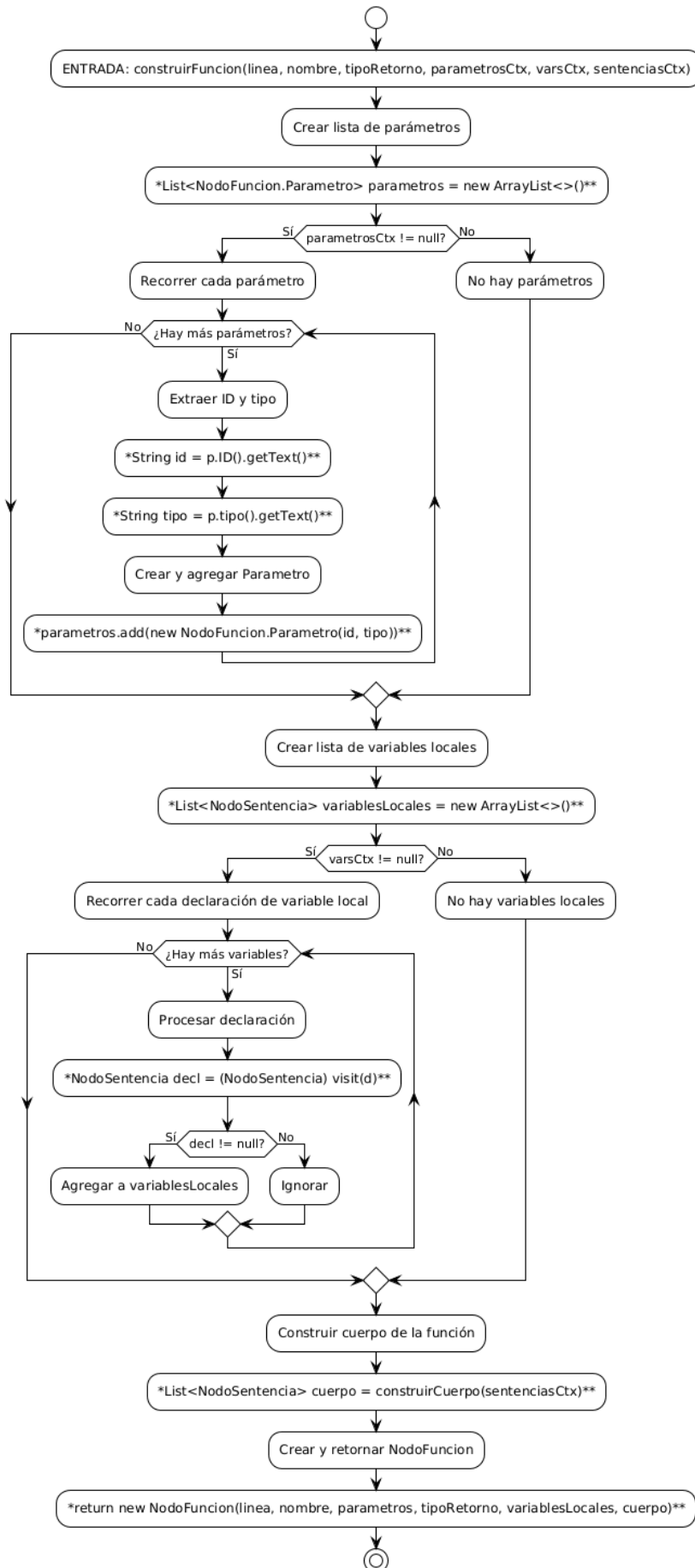


Sub-flujo: construirMapaCampos() - Mapa de Campos de Struct



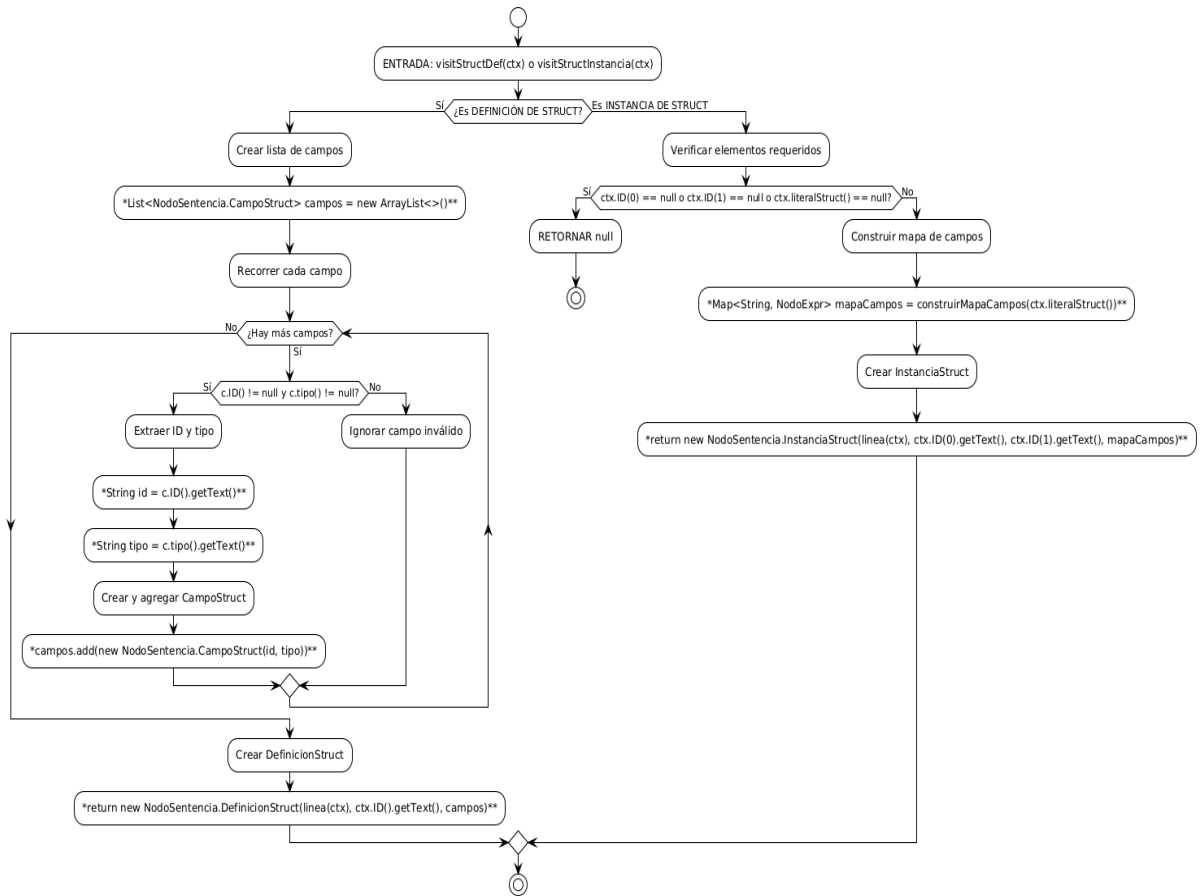


Flujo: construirFuncion() - Declaración Completa





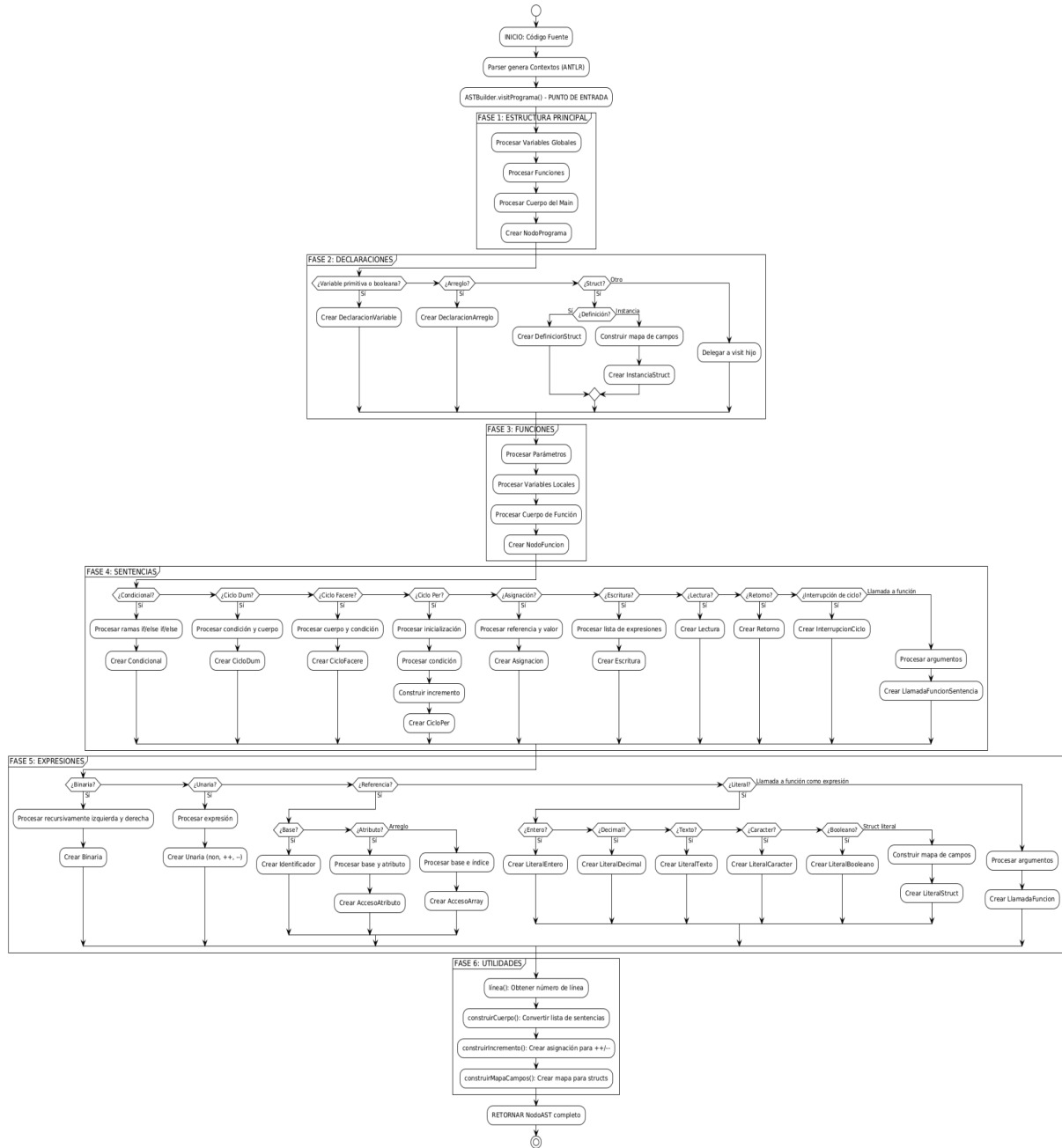
Flujo: Manejo de Structs - Definición e Instancia





flujo general

Diagrama General: ASTBuilder - Flujo Completo





CompileService

Es el servicio central de compilación del proyecto.

Se encarga de coordinar todo el proceso: análisis léxico, sintáctico, construcción del AST, validación semántica, coloreado y traducción a PigLatin.

Expone dos métodos principales:

analizar(String codigo) -> realiza el análisis completo del código fuente.

traducir(NodoPrograma programa) -> convierte el AST en código PigLatin.

Métodos principales

analizar(String codigo)

Punto de entrada para analizar un código fuente.

Verifica si el código está vacío y maneja errores globales como StackOverflowError o excepciones internas.

Llama a **analizarInterno** para ejecutar el pipeline de compilación.

Devuelve un objeto **ResultadoCompilacion** con:

Estado de éxito o error.

AST generado.

Tabla de símbolos.

Errores léxicos, sintácticos o semánticos.

Pasos de pila .

traducir(NodoPrograma programa)

Recibe un AST ya construido.

Llama al método **toPigLatin** de **NodoPrograma** para generar la traducción.

Devuelve el código traducido como String.

Maneja errores de traducción y devuelve mensajes claros si algo falla.

Método interno: **analizarInterno(String codigo)**

Este método implementa el pipeline completo de compilación:

- Lexer
- Usa LatinusLexer para tokenizar el código.
- Captura errores léxicos con un BaseErrorListener.
- Si hay errores, se detiene y devuelve un ResultadoCompilacion con los errores.
- Parser
- Usa LatinusParser para construir el árbol sintáctico.
- Añade un PilaListener para registrar pasos de pila.
- Captura errores sintácticos.
- Si hay errores, se devuelve un resultado con los errores.
- Construcción del AST
- Usa ASTBuilder para transformar el árbol sintáctico en un NodoPrograma.
- Si falla, devuelve un error indicando que el AST no pudo construirse.
- Validación semántica
- Usa ValidadorSemantico para verificar tipos, variables y coherencia del programa.
- Devuelve una lista de ErrorSemantico.
- Si hay errores, se reportan en el resultado.
- Coloreado



- Vuelve a ejecutar el parser y aplica ASTColor para generar un coloreado del código.
- Produce una lista de TextoColoreado para resaltar sintaxis en la interfaz.
- Resultado final

Si todo es correcto, devuelve un **ResultadoCompilacion** con éxito, el AST, la tabla de símbolos y el coloreado.

Si hay errores en cualquier etapa, devuelve un resultado con los errores correspondientes.

Utilidad

Manejo de errores robusto: captura excepciones en cada etapa y devuelve mensajes claros.

Integración modular: conecta lexer, parser, ASTBuilder, validador semántico y coloreado en un flujo ordenado.

En resumen

La clase CompileService es el motor de compilación. Orquesta todas las fases:

Análisis léxico y sintáctico con ANTLR.

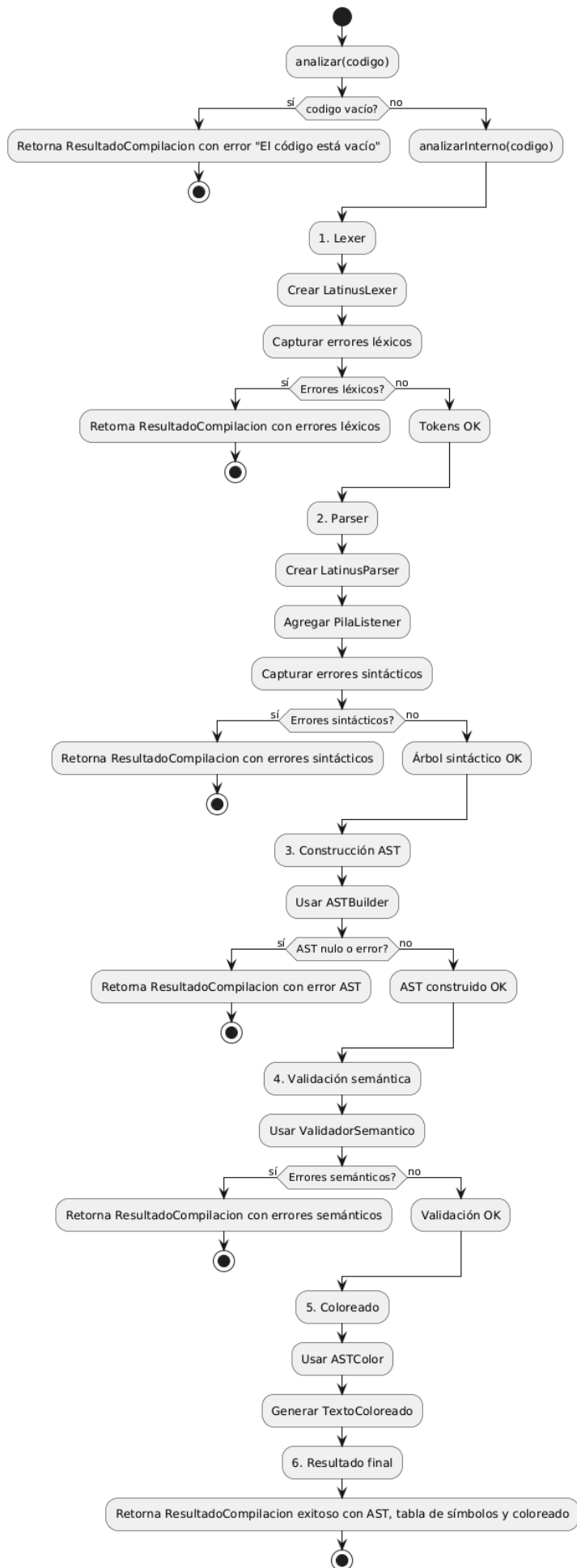
Construcción del AST con ASTBuilder.

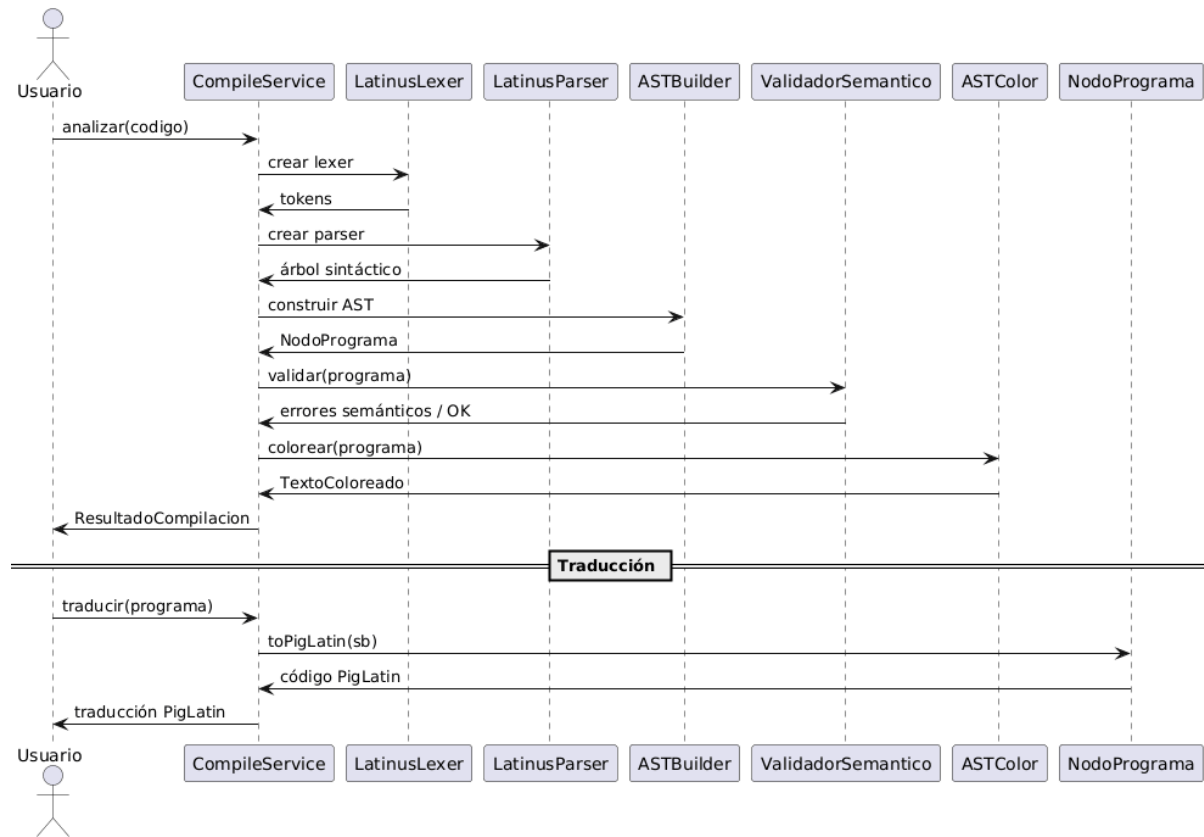
Validación semántica con ValidadorSemantico.

Coloreado con ASTColor.

Traducción final a PigLatin.

Es el punto central que asegura que el código fuente pase por todas las etapas necesarias para ser analizado y transformado correctamente.







ValidadorSemantico

Es el módulo encargado de la validación semántica del programa.

Se asegura de que las declaraciones, tipos, estructuras y flujo de control sean correctos y coherentes.

Utiliza una tabla de símbolos para registrar variables, funciones y estructuras, y una lista de errores semánticos para reportar inconsistencias.

Coordina varios validadores especializados:

ValidadorDeclaraciones

ValidadorAlcance

ValidadorTipos

ValidadorEstructuras

ValidadorFlujo

Métodos principales

validar(NodoPrograma programa)

Punto de entrada para validar un programa completo.

Etapas:

Validar variables globales.

Validar funciones (declaración, parámetros, cuerpo).

Validar el bloque main.

Devuelve la lista de errores semánticos encontrados.

validarDeclaracion(NodoSentencia sentencia)

Valida declaraciones de variables, arreglos y structs.

Si hay valores iniciales, valida también las expresiones asociadas.

Usa ValidadorDeclaraciones y ValidadorEstructuras.

validarFuncion(NodoFuncion funcion)

Declara la función en la tabla de símbolos.

Registra parámetros y variables locales.

Valida el cuerpo de la función con control de flujo (ValidadorFlujo).

Maneja el scope de la función (entrada y salida).

validarMain(List<NodoSentencia> main)

Similar a validarFuncion, pero para el bloque principal.

No requiere tipo de retorno.

Valida sentencias dentro del main.

validarSentencia(NodoSentencia sentencia)

Determina el tipo de sentencia y aplica la validación correspondiente:

Declaraciones.

Flujo (retorno, break, continue).

Asignaciones.

Escritura y lectura.

Ciclos (dum, facere, per).

Condicionales.

Llamadas a funciones.

Validaciones específicas

Asignaciones: comprueba compatibilidad de tipos entre referencia y valor.

Lectura: verifica que la variable exista en la tabla de símbolos.

Escritura: valida cada expresión a imprimir.

Retorno: valida el valor retornado.



Ciclos: controlan scopes y condiciones.

Condicionales: validan condiciones y cuerpos de ramas.

Llamadas a funciones: validan argumentos y tipos.

validarExpresion(NodoExpr expr)

Valida alcance y tipos de expresiones.

Comprueba acceso a atributos de structs.

Valida recursivamente subexpresiones binarias, unarias, arrays y llamadas a funciones.

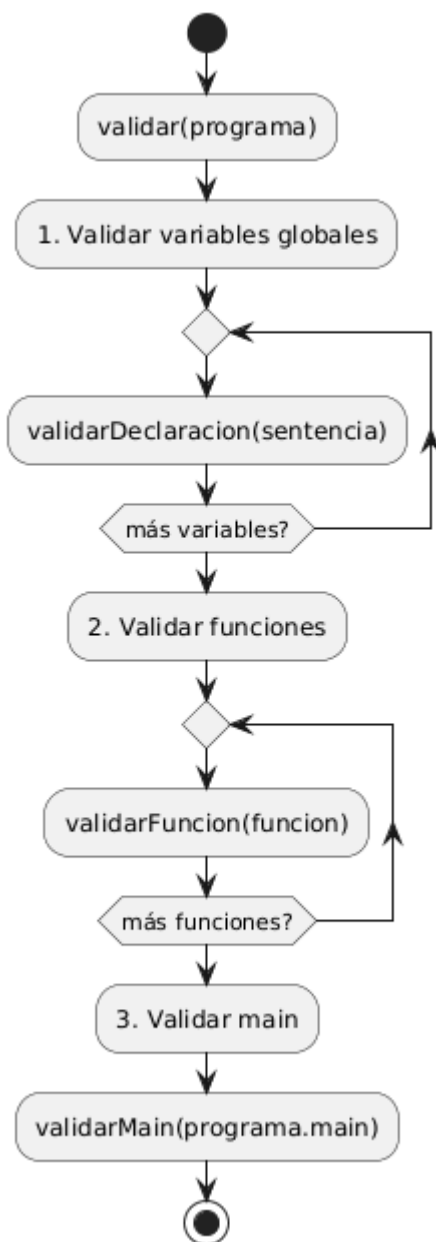
Utilidad

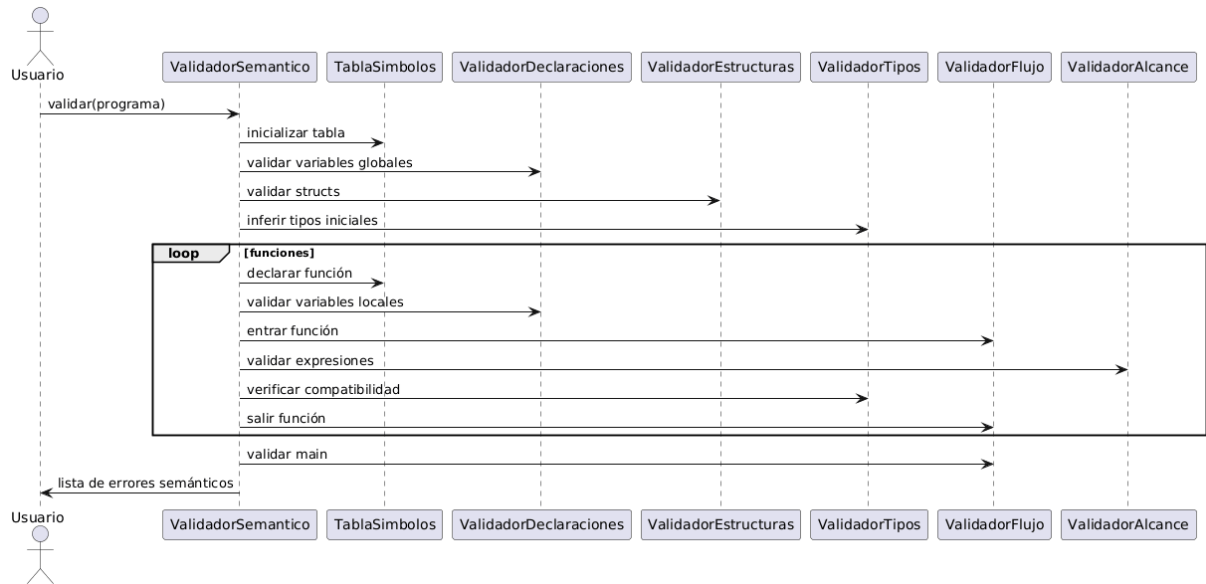
Tabla de símbolos (TablaSimbolos): almacena variables, funciones y parámetros con sus tipos y alcances.

Errores semánticos (ErrorSemantico): lista acumulativa de inconsistencias detectadas.

Scopes: cada función, ciclo o condicional abre un nuevo ámbito para las variables.

Validadores especializados: cada aspecto semántico se delega a un validador concreto.





ValidadorAlcance

Rol principal: Se encarga de validar el alcance de las expresiones en el programa, asegurando que las variables utilizadas estén declaradas en la tabla de símbolos.

Colaboradores:

TablaSimbolos: consulta si las variables existen.

ErrorSemantico: registra errores cuando se usan variables no declaradas.

Métodos principales:

validar(NodoExpr expresion):

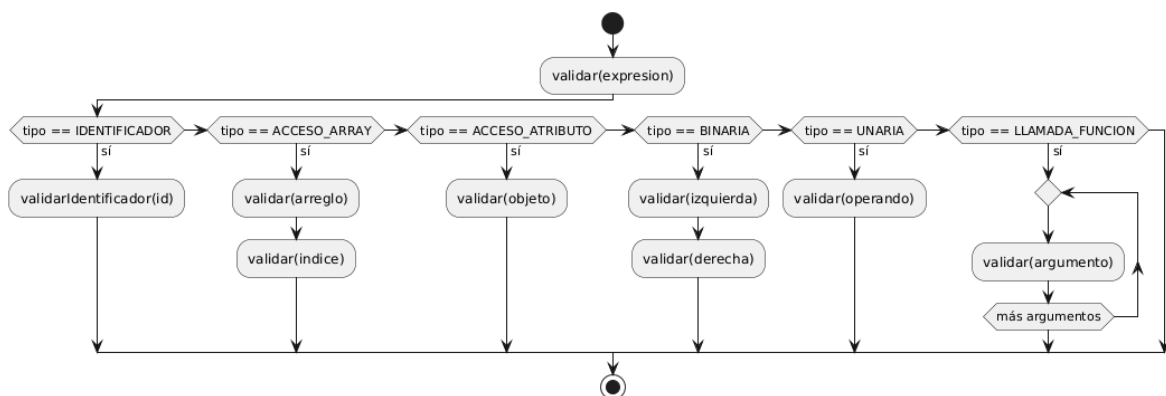
Recorre recursivamente la expresión según su tipo (IDENTIFICADOR, ACCESO_ARRAY, ACCESO_ATRIBUTO, BINARIA, UNARIA, LLAMADA_FUNCION).

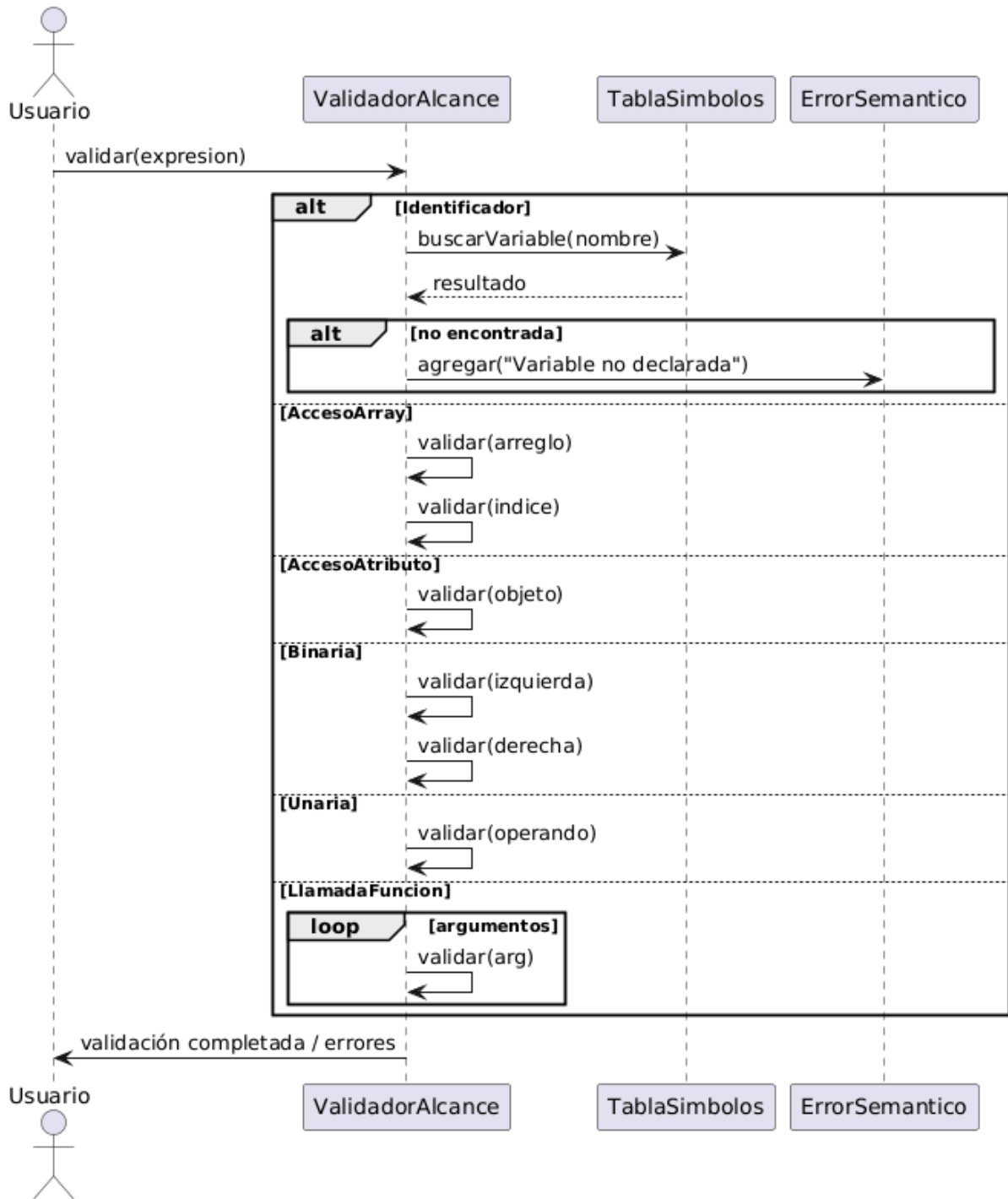
Llama a subvalidaciones para cada caso.

validarIdentificador(NodoExpr.Identificador id):

Comprueba si la variable existe en la tabla de símbolos.

Si no existe, agrega un ErrorSemantico indicando "Variable no declarada".







ValidadorDeclaraciones

Rol principal: Se encarga de validar las declaraciones de variables, arreglos y estructuras (structs) en el programa.

Colaboradores:

TablaSimbolos: registra variables, arreglos y structs en el ámbito correspondiente.

ErrorSemantico: almacena errores cuando hay duplicados o declaraciones inválidas.

Métodos principales:

validar(NodoSentencia sentencia):

Determina el tipo de sentencia (DECLARACION_VARIABLE, DECLARACION_ARREGLO, DEFINICION_STRUCT).

Llama al método específico de validación.

validarDeclaracionVariable:

Declara una variable en la tabla de símbolos.

Si ya existe, registra un error semántico.

validarDeclaracionArreglo:

Declara un arreglo con tipo y tamaño.

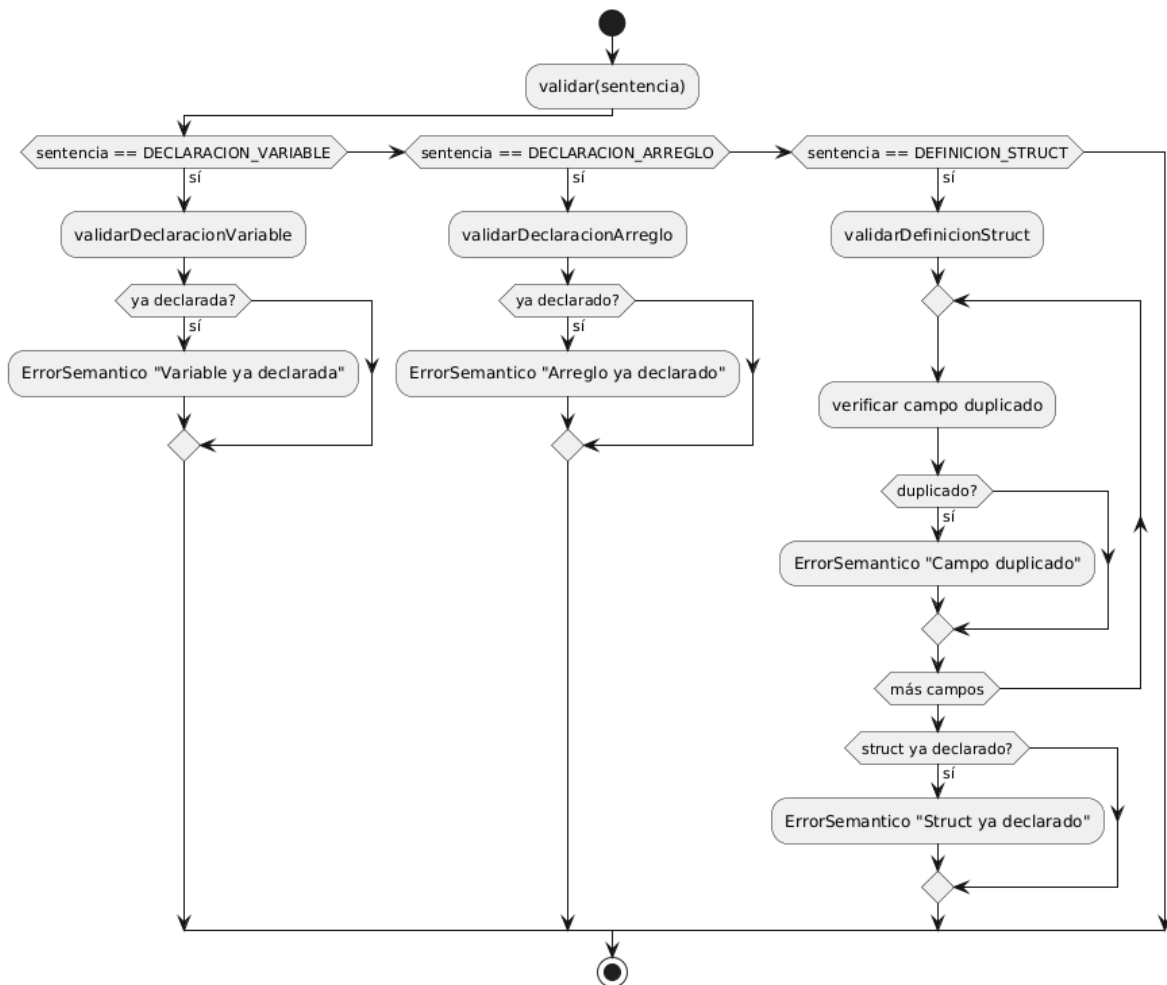
Si ya existe, registra un error semántico.

validarDefinicionStruct:

Recorre los campos de un struct, detecta duplicados y los registra.

Declara el struct en la tabla de símbolos.

Si ya existe, registra un error semántico.





ValidadorEstructuras

Rol principal: Se encarga de validar el uso de estructuras (structs) en el programa.

Colaboradores:

TablaSimbolos: consulta y registra definiciones de structs e instancias.

ErrorSemantico: almacena errores cuando se usan structs no declarados, campos inexistentes o tipos incorrectos.

ValidadorTipos: infiere tipos de expresiones para comprobar compatibilidad.

Métodos principales:

validar(NodoSentencia sentencia):

Si la sentencia es una instancia de struct, llama a validarInstanciaStruct.

validarAccesoAtributo(NodoExpr.AccesoAtributo acceso):

Comprueba que el objeto sea de tipo struct.

Verifica que el campo accedido exista en la definición del struct.

validarInstanciaStruct(NodoSentencia.InstanciaStruct instancia):

Comprueba que el struct esté declarado.

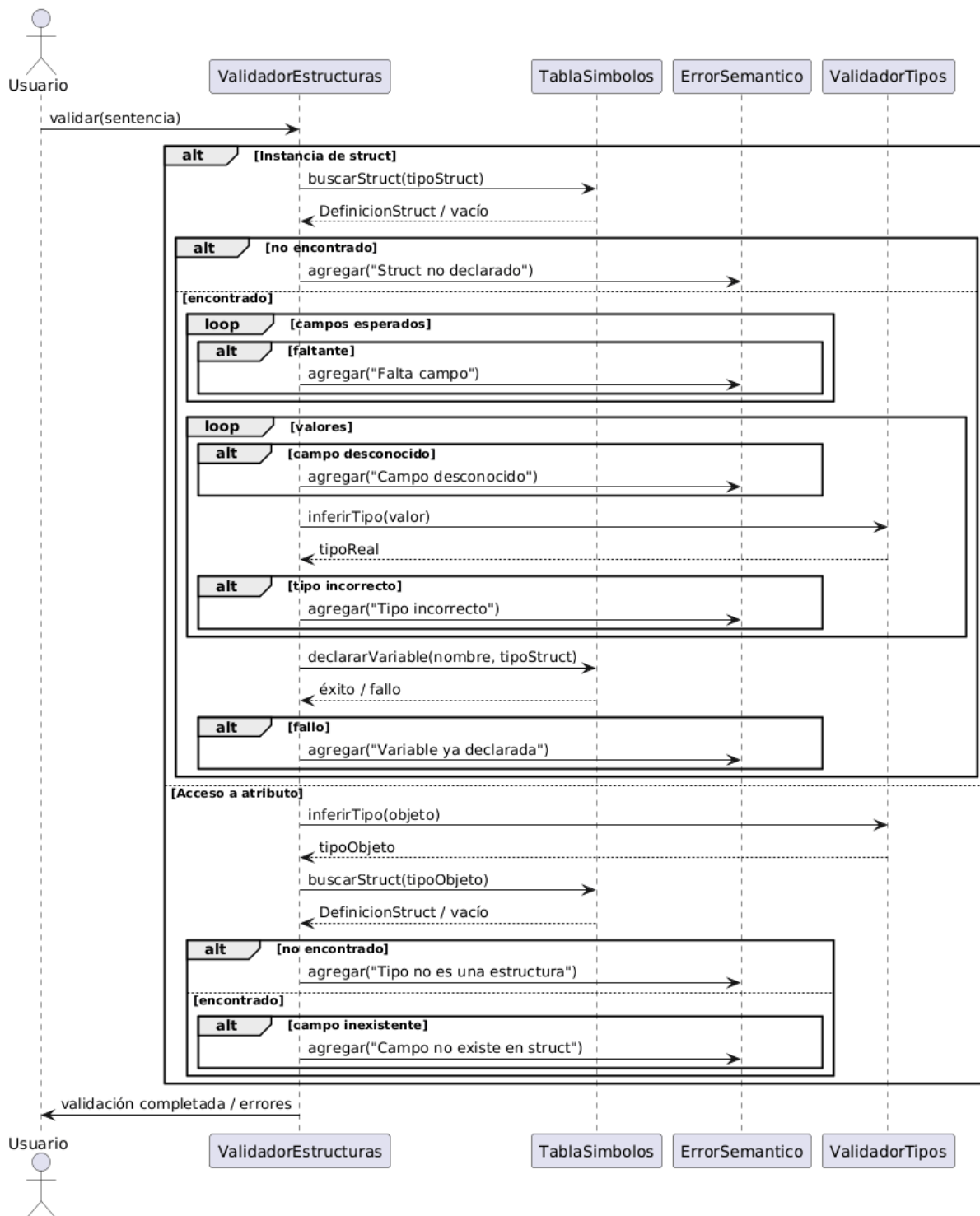
Verifica que todos los campos esperados estén presentes.

Detecta campos extra no definidos.

Valida que los valores asignados a cada campo coincidan con el tipo esperado.

Declara la instancia en la tabla de símbolos.

Si ya existe una variable con ese nombre, registra un error.





ValidadorFlujo

Rol principal: Se encarga de validar el flujo de control en el programa, asegurando que las sentencias de interrupción (break, continue) y de retorno (return) se usen en contextos válidos.

Colaboradores:

ErrorSemantico: registra errores cuando se usan sentencias fuera de lugar.

Atributos principales:

dentroDeCiclo: indica si la validación está dentro de un ciclo.

dentroDeFuncion: indica si la validación está dentro de una función.

tipoRetornoEsperado: tipo de retorno esperado en la función actual.

Métodos principales:

entrarCiclo / salirCiclo: marcan el inicio y fin de un ciclo.

entrarFuncion / salirFuncion: marcan el inicio y fin de una función, registrando el tipo de retorno esperado.

validar(NodoSentencia sentencia):

Si la sentencia es interrupción de ciclo, llama a validarInterrupcion.

Si es retorno, llama a validarRetorno.

validarInterrupcion:

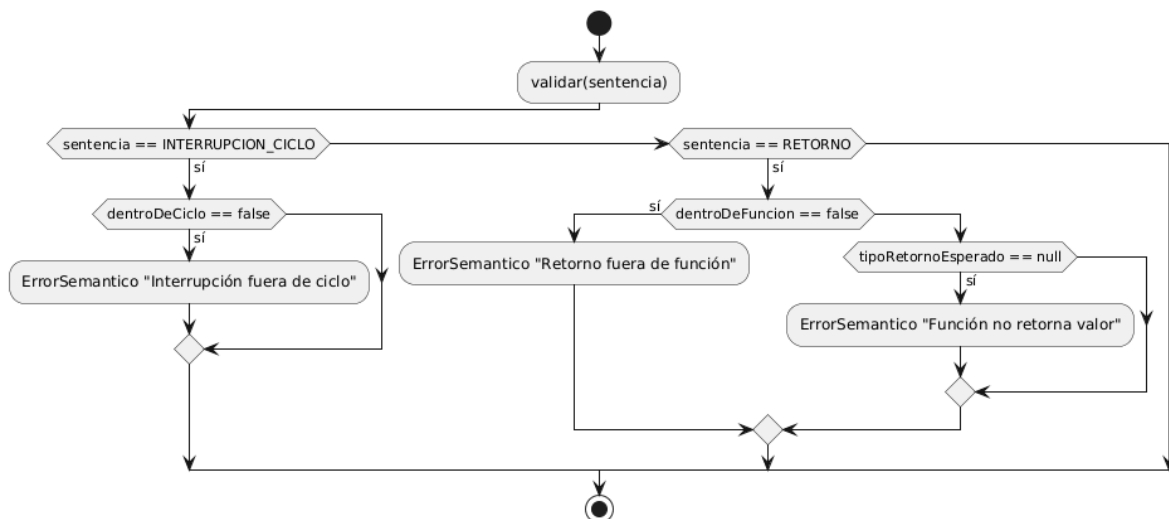
Verifica que la sentencia esté dentro de un ciclo.

Si no, genera un error semántico.

validarRetorno:

Verifica que la sentencia esté dentro de una función.

Si no, genera un error semántico.





ValidadorTipos

Rol principal: Se encarga de inferir y validar tipos de expresiones en el programa, asegurando la compatibilidad entre operaciones, accesos y llamadas a funciones.

Colaboradores:

TablaSimbolos: consulta tipos de variables, arreglos, structs y funciones.

ErrorSemantico: registra errores cuando se usan tipos incompatibles o no declarados.

Atributos principales:

JERARQUIA: mapa que define la jerarquía de tipos (textum > decimalis > numerus > littera > booleano).

tabla: referencia a la tabla de símbolos.

errores: lista acumulativa de errores semánticos.

Métodos principales:

inferirTipo(NodoExpr expr): determina el tipo de una expresión según su nodo (literales, identificadores, operaciones, accesos, llamadas).

inferirBinaria: valida operaciones binarias, aplica jerarquía de tipos y asegura que textum solo se concatene con +.

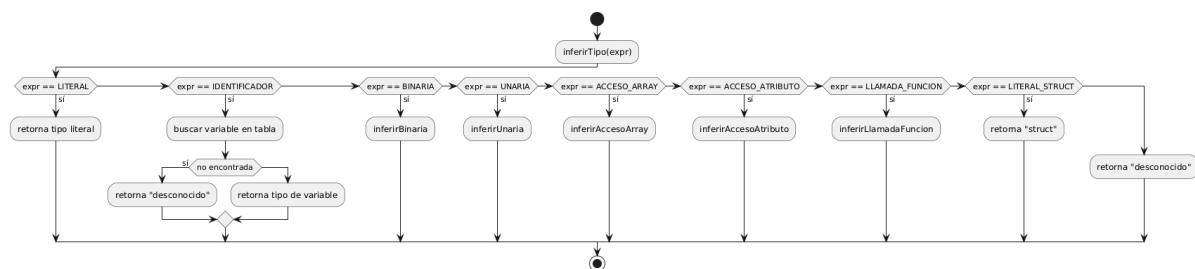
inferirUnaria: valida operaciones unarias (non, ++, --) asegurando que se apliquen a tipos correctos.

inferirAccesoArray: valida que el índice sea numerus y devuelve el tipo de los elementos del arreglo.

inferirAccesoAtributo: valida que el objeto sea un struct y que el campo exista.

inferirLlamadaFuncion: valida existencia de la función, número de argumentos y compatibilidad de tipos.

sonCompatibles: determina si un tipo origen puede asignarse a un tipo destino según la jerarquía.





TablaSimbolos

Rol principal: Es la estructura central de gestión de símbolos en el compilador.

Administra variables, arreglos, structs y funciones.

Controla los alcances (scopes) y el registro de símbolos para depuración y visualización.

Colaboradores:

Usada por los validadores semánticos (ValidadorDeclaraciones, ValidadorTipos, ValidadorEstructuras, etc.).

Proporciona información a la interfaz de usuario (UI) mediante getRegistroCompleto.

Elementos principales:

Categoría: Enum que clasifica símbolos (VARIABLE, ARREGLO, PARAMETRO, STRUCT_INSTANCIA, STRUCT_DEF, FUNCION).

Registros internos:

Parametro: nombre y tipo de parámetros de funciones.

SimboloVariable: nombre, tipo, si es arreglo y tamaño.

DefinicionStruct: nombre y campos de un struct.

DefinicionFuncion: nombre, parámetros y tipo de retorno.

EntradaSimbolo: registro detallado de cada símbolo (id, nombre, categoría, tipo, ámbito, alcance).

Scope: clase interna que mantiene variables y structs por bloque.

Pila de scopes: Deque<Scope> que gestiona el contexto actual.

Funciones: Map<String, DefinicionFuncion> para almacenar funciones declaradas.

Registro completo: List<EntradaSimbolo> para auditoría y depuración.

Métodos principales:

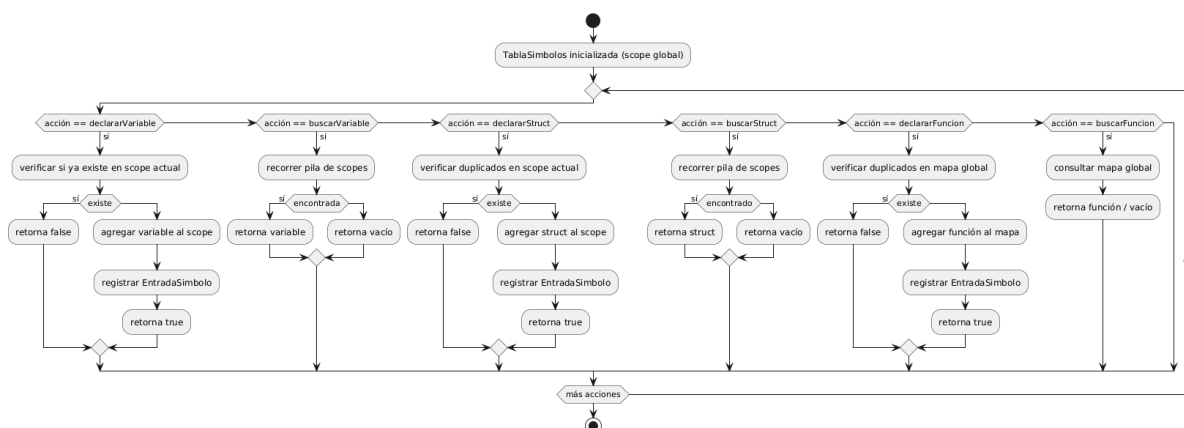
Gestión de scopes: entrarScope, salirScope, ambitoActual, alcanceActual.

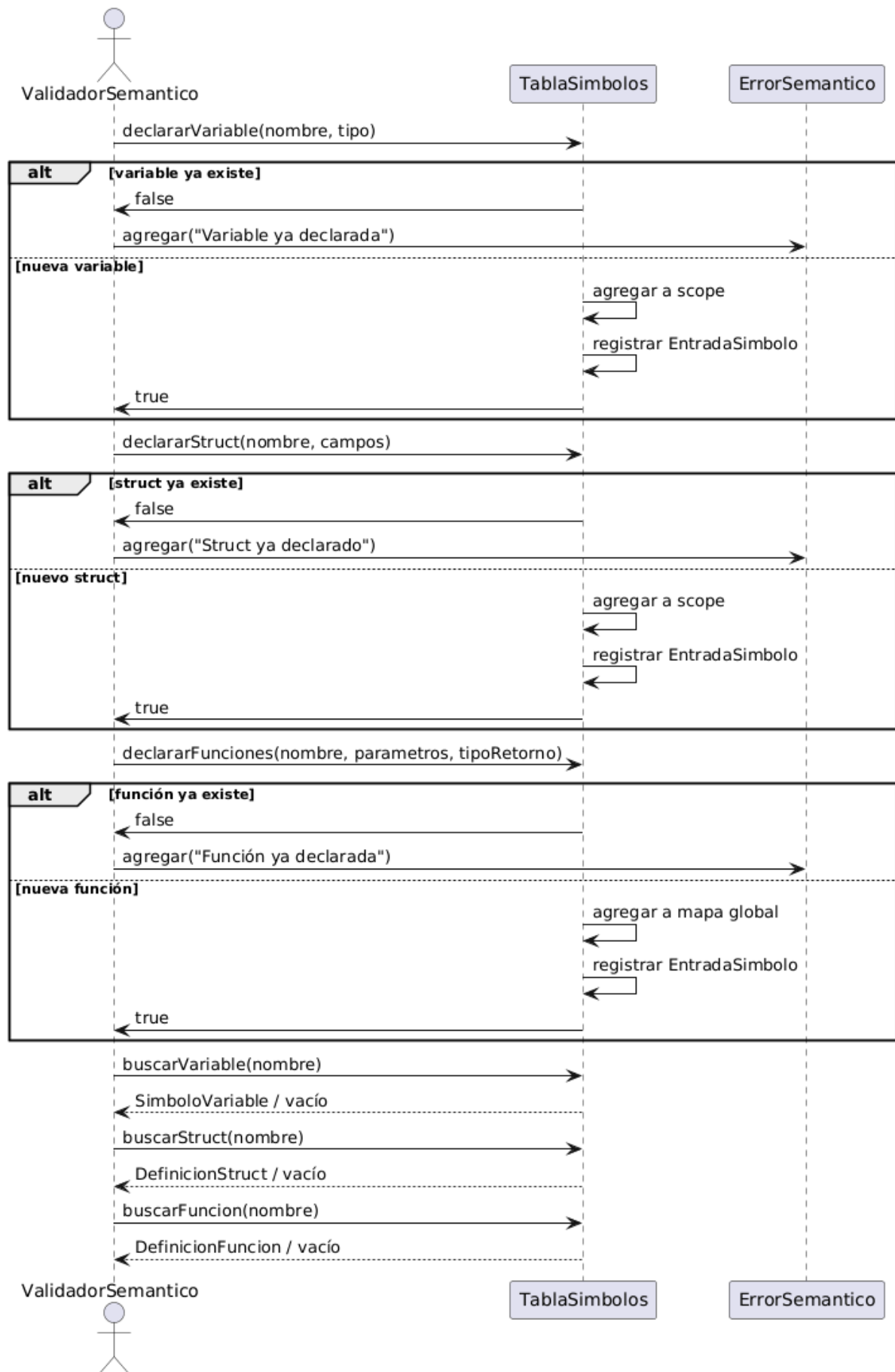
Variables y arreglos: declararVariable, buscarVariable.

Structs: declararStruct, buscarStruct.

Funciones: declararFunciones, buscarFuncion.

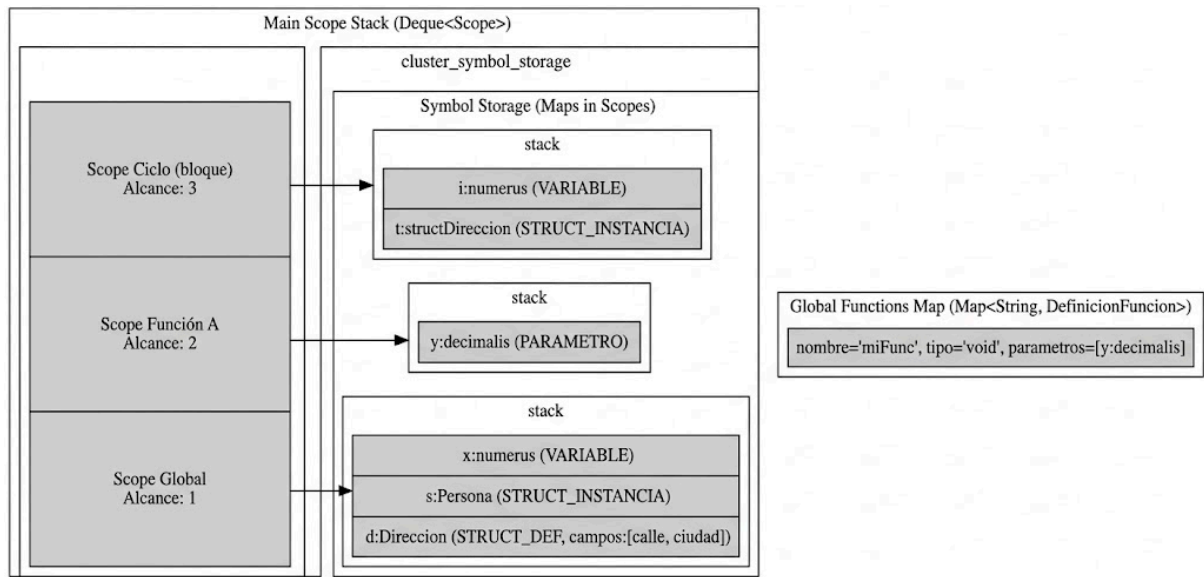
Lectura para UI: getFunciones, getRegistroCompleto.







graficas visuales



PilaListener

PasoPila

Rol: Representa un paso individual dentro del análisis sintáctico con pila.

Campos:

numero: número de paso (contador).

operacion: tipo de operación (SHIFT, REDUCE, ACCEPT).

simbolo: el símbolo procesado (terminal o no terminal).

simbolosReducidos: lista de símbolos que fueron reducidos (en caso de REDUCE).

pila: snapshot del estado actual de la pila en ese paso.

descripcion: texto descriptivo de la operación realizada.

Uso: Es un registro histórico que permite reconstruir cómo se procesó la entrada paso a paso.

TipoOperacion

Rol: Enum que define los tipos de operaciones posibles en el análisis con pila.

Valores:

SHIFT: insertar un símbolo en la pila.

REDUCE: aplicar una regla gramatical y reemplazar símbolos por un no terminal.

ACCEPT: aceptar la entrada completa (cuando se reduce la regla raíz).



PilaListener

Rol: Listener que extiende `LatinusParserBaseListener` y se encarga de registrar cada paso del análisis sintáctico.

Atributos:

pila: estructura `Deque<String>` que representa la pila de símbolos.

pasos: lista de `PasoPila` que guarda el historial de operaciones.

contador: número de pasos realizados.

Métodos principales:

`visitTerminal(TerminalNode node):`

Cada vez que se encuentra un terminal, se hace un SHIFT.

Se agrega el símbolo a la pila y se registra un `PasoPila`.

`exitEveryRule(ParserRuleContext ctx):`

Cuando se completa una regla, se hace un REDUCE.

Se sacan los símbolos de la pila, se reemplazan por el no terminal.

Si es la regla raíz, se marca como ACCEPT.

Se registra un `PasoPila` con la descripción de la reducción.

`nombreRegla:` obtiene el nombre de la regla a partir de la clase del contexto.

`snapshot:` devuelve una copia invertida de la pila para mostrar su estado actual.

`getPasos:` retorna la lista completa de pasos registrados.

